
Reinforcement Learning

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io

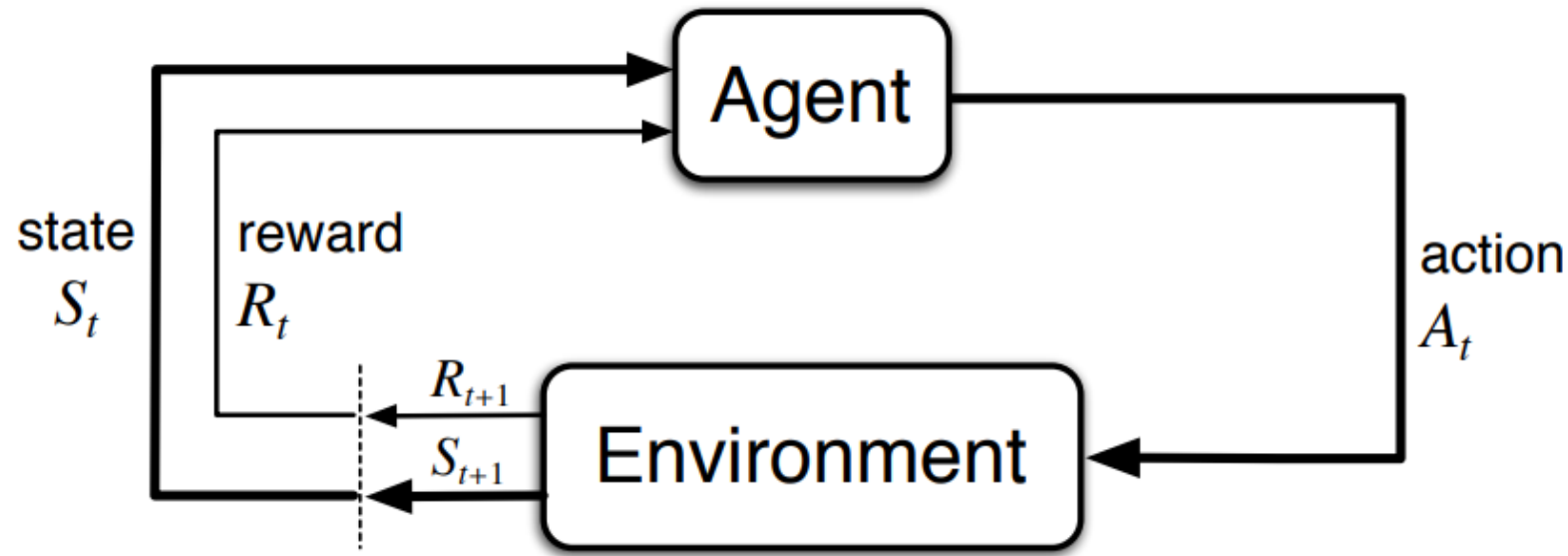
This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

References

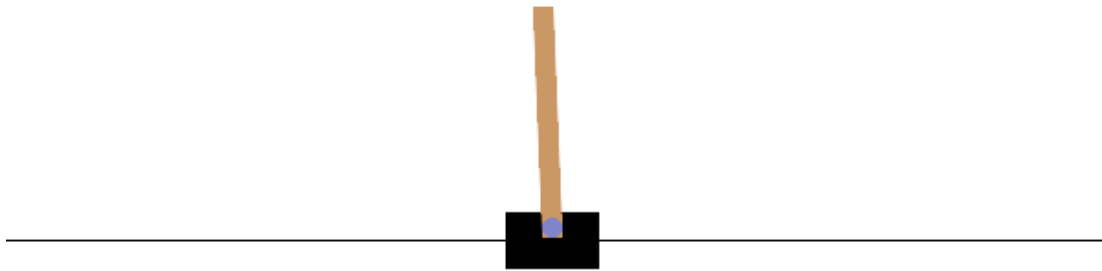
1. Dedicated book. Reinforcement Learning An Introduction ed. 2020. Sutton and Barto.
2. Short overview of RL: <https://faculty.washington.edu/sbrunton/databookRL.pdf>
3. An Introduction to Deep Reinforcement Learning. Lavet et. al. <https://arxiv.org/pdf/1811.12560.pdf>
4. Policy Gradient Methods Summaries: <https://lilianweng.github.io/lil-log/2018/04/08/policy-gradient-algorithms.html#Proof-of-Policy-Gradient-Theorem>

Problem Overview

Problem Overview



- From current state S_t , agent interacts with the environment via action A_t , and perceives a next state S_{t+1} and reward R_{t+1} .
- An agent uses a **policy** π , which is a function that takes in a state and outputs an action.
- **Goal of RL:** find an **optimal policy** π^* that maximizes the cumulative reward.



- **Objective:** Balance a pole on top of a movable cart
- **State:** angle, angular speed, position, horizontal velocity
- **Action:** horizontal force applied on the cart
- **Reward:** 1 at each time step if the pole is upright

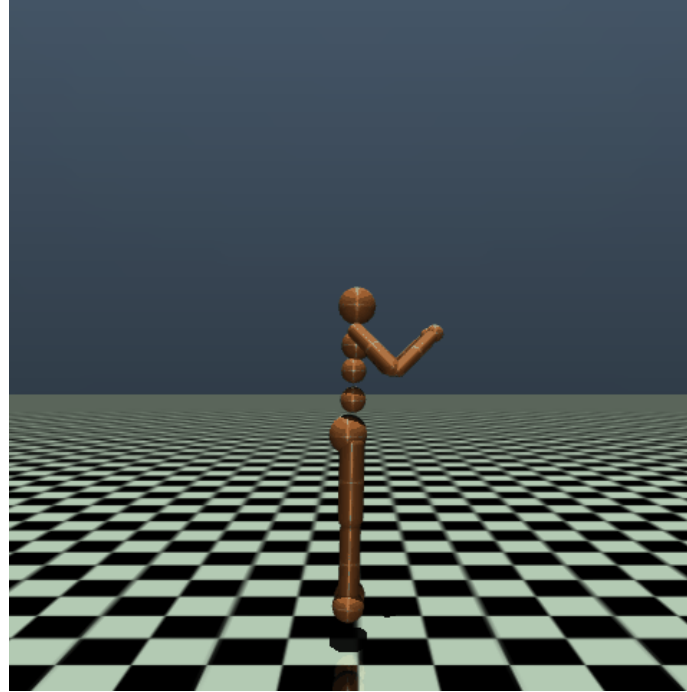
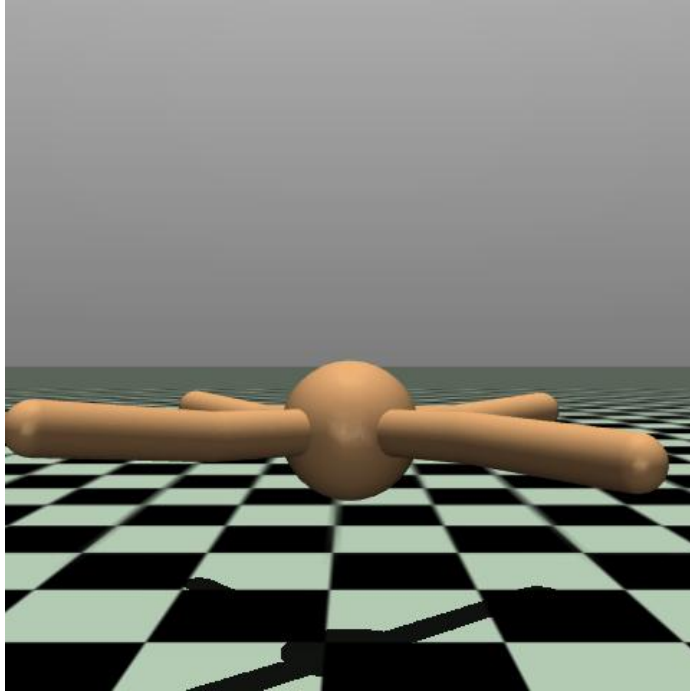
https://gymnasium.farama.org/environments/classic_control/cart_pole/

RL Examples – Bipedal Walker



- **Objective:** Get to far-right end of environment.
- **State:** hull angle speed, angular velocity, horizontal speed, vertical speed, position of joints and joints angular speed, legs contact with ground, and 10 lidar rangefinder measurements.
- **Action:** motor speed values in the $[-1, 1]$ range for each of the 4 joints at both hips and knees.
- **Reward:** Reward is given for moving forward, totaling 300+ points up to the far end.

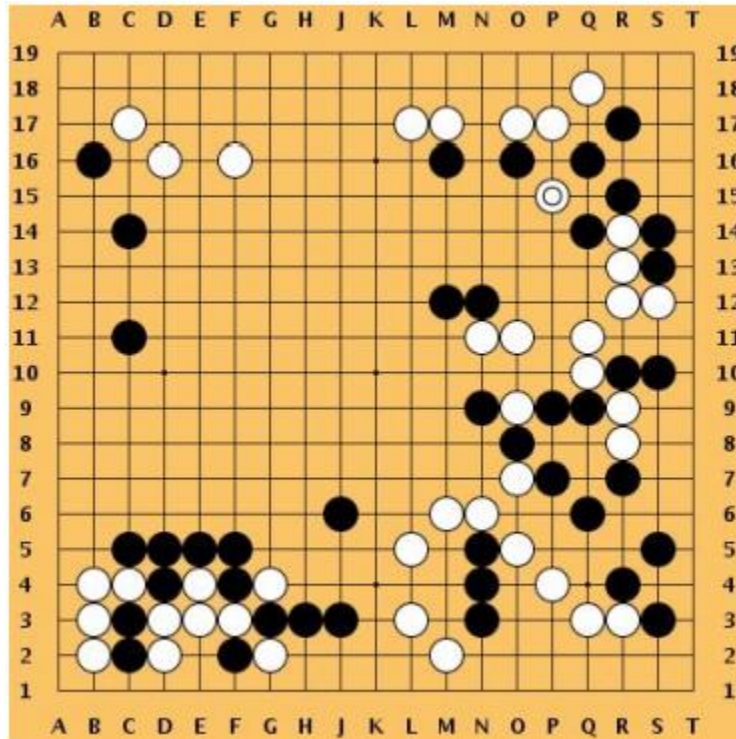
https://gymnasium.farama.org/environments/box2d/bipedal_walker/



- **Objective:** Make the robot move forward
- **State:** Angle and position of the joints
- **Action:** Torques applied on joints
- **Reward:** 1 at each time step upright + forward movement.

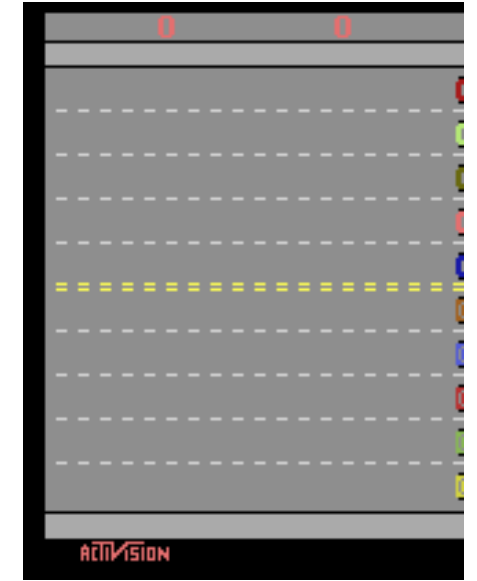
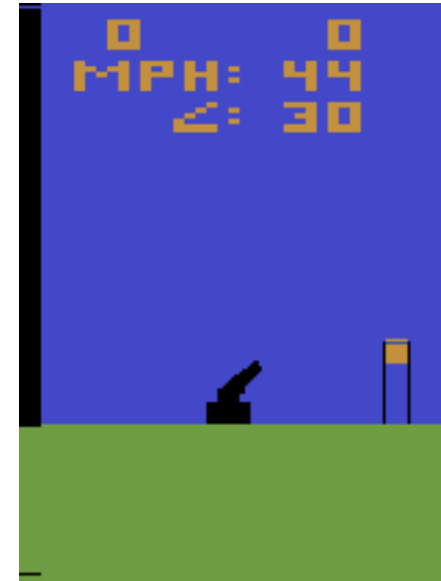
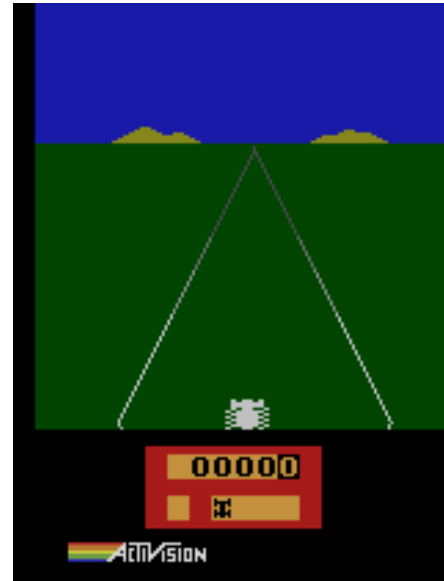
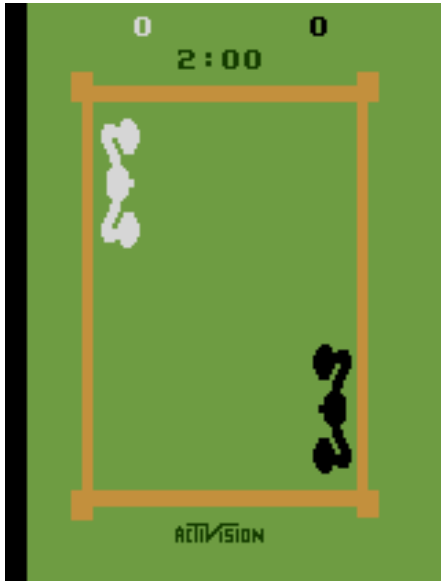
<https://gymnasium.farama.org/environments/mujoco/>

RL Examples – Board Games like Go



- **Objective:** Win game
- **State:** board position
- **Action:** position pieces
- **Reward:** 1 for win game, -1 for loss.

RL Examples – Atari Games



- **Objective:** Win game
- **State:** Raw pixel inputs of the game state
- **Action:** game controls
- **Reward:** game score

<https://gymnasium.farama.org/environments/atari/>

gym-trading-env



Metrics

Market Return

62.35%

Portfolio Return

-24.42%

Parameters

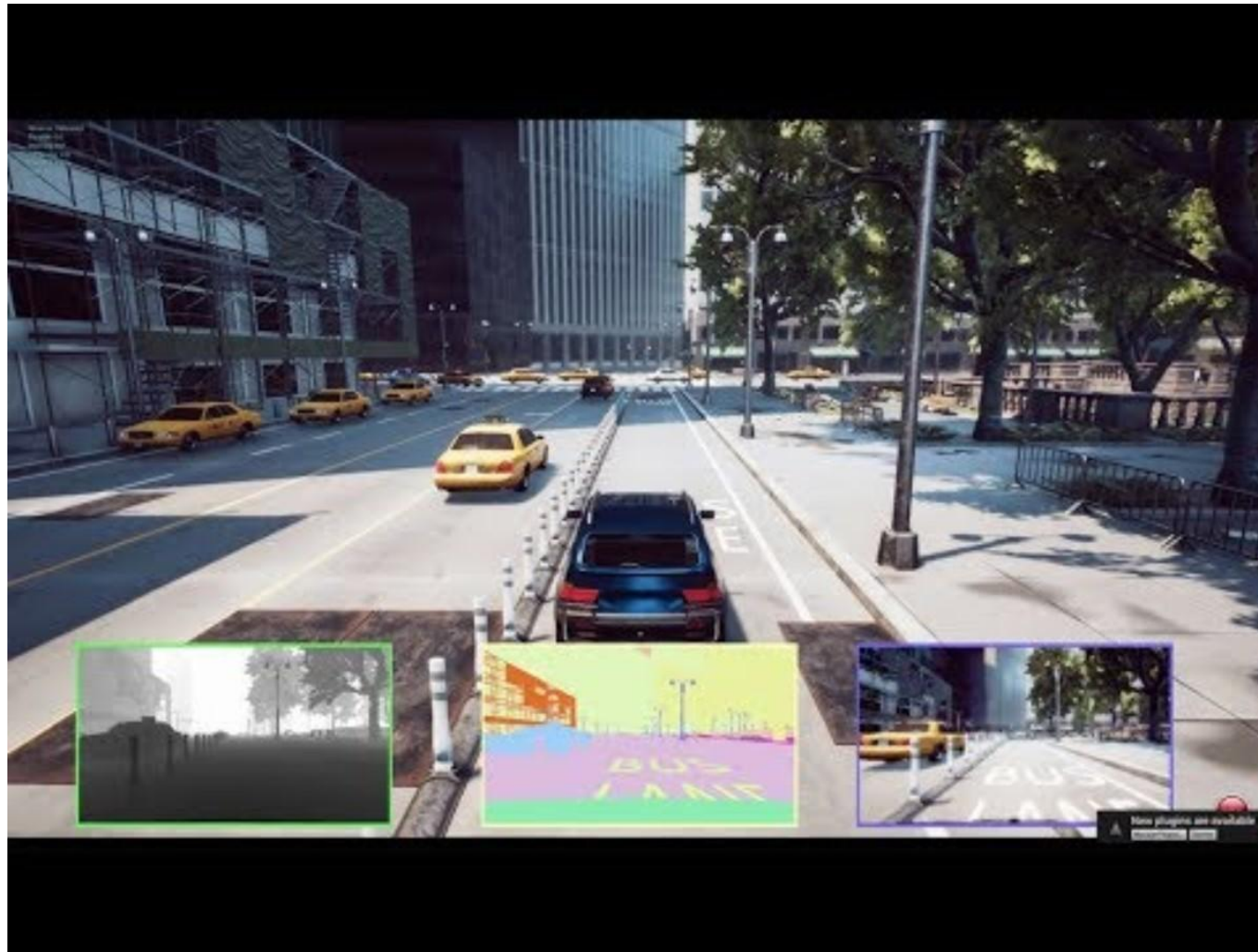
Select render

bybit-BTCUSDT-1h.pkl_2023-04-25_11-...

by Clément Perroud, 2023

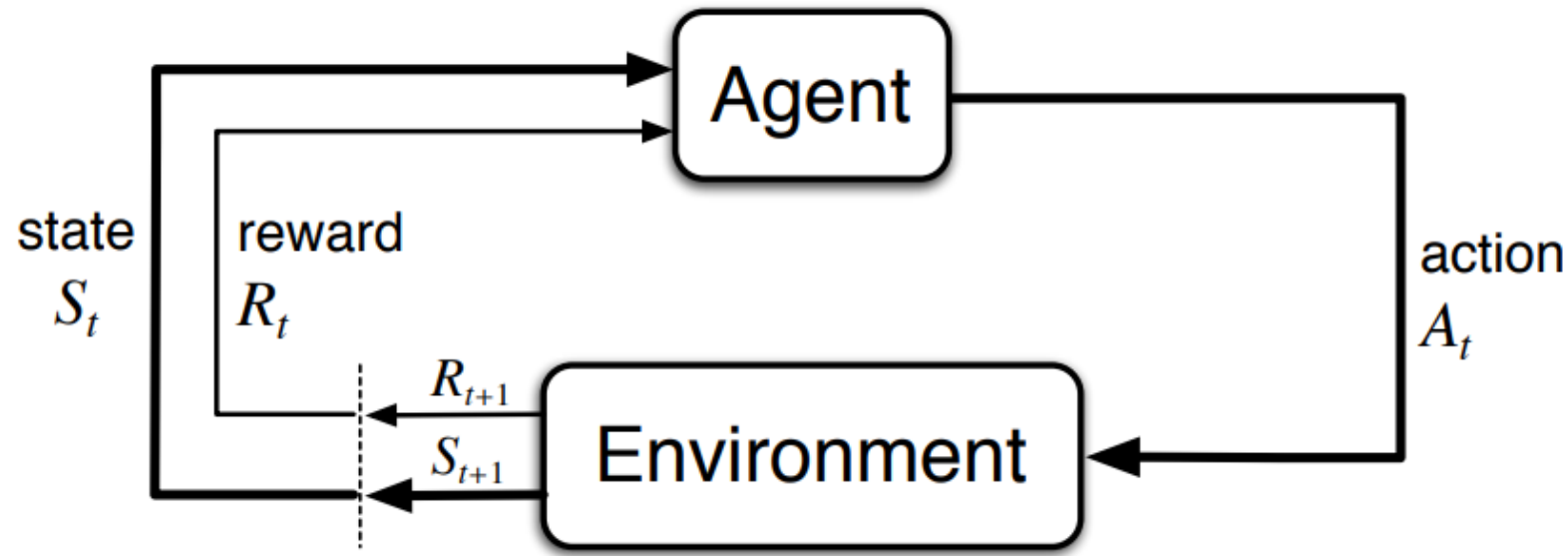
<https://gym-trading-env.readthedocs.io/en/latest/>

RL Examples – Self-driving cars



<https://github.com/microsoft/AirSim/>

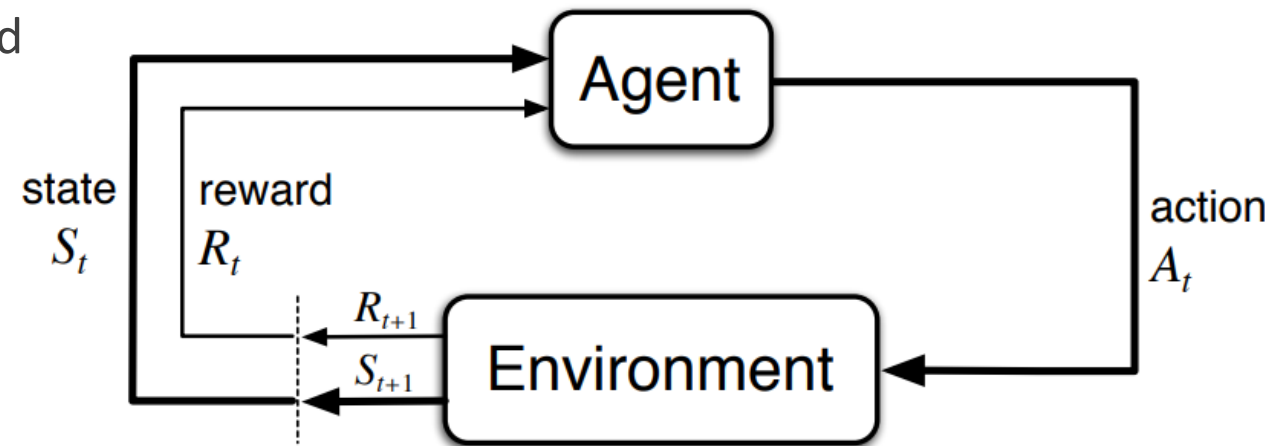
Problem Overview



- From current state S_t , agent interacts with the environment via action A_t , and perceives a next state S_{t+1} and reward R_{t+1} .
- An agent uses a **policy** π , which is a function that takes in a state and outputs an action.
- **Goal of RL:** find an **optimal policy** π^* that maximizes the cumulative reward.

Markov Decision Process (MDP)

- The **Markov Property**: the future state of a process depends *only* on the current state, not on the sequence of states that preceded it.
- A **Markovian decision process (MDP)** is a formal framework for sequential decision-making under uncertainty that satisfies the Markov property.
- This assumption makes the problem computationally easier. We only need to reason about the current state, not remember the entire trajectory.
- RL is a sequential process over time that can model as an MDP.
- In RL, we typically make the assumption that the process is an MDP. The next state S_{t+1} and reward R_{t+1} depend only on previous S_t and A_t



Problem Overview – Formalized [1]

- In RL, a *common* assumption is that the problem is a **Markovian decision process (MDP)** [next state and reward depends only on previous state and previous action only; this simplifies the problem]

$$P(S_{t+1}, R_{t+1} | S_t, A_t, S_{t-1}, A_{t-1}, \dots, S_0, A_0) = P(S_{t+1}, R_{t+1} | S_t, A_t)$$

- We formalize the problem as $(\mathcal{S}, \mathcal{A}, P)$
 - $\mathcal{S}$ is set of possible environment states
 - $\mathcal{A}$ is set of possible agent actions
 - $P(s', r | s, a)$ is a probability model for transitions and rewards given some state and action.
 - γ is discount factor for rewards. Between 0 and 1.
- Expected *reward* is then defined as: $r(s, a) \doteq \mathbb{E}[R_{t+1} | S_t = s, A_t = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r | s, a)$

Notice that R_t is random variable.

$$r(s, a, s') \doteq \mathbb{E}[R_{t+1} | S_t = s, A_t = a, S_{t+1} = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r | s, a)}{p(s' | s, a)}$$

- Goal** is to find a policy π^* that maximizes expected **return** $\mathbb{E}(\sum \gamma^k R_k | S_0 = s)$

Problem Overview – Formalized [2]

- We denote a policy as a probability function $\pi(a \mid s) \rightarrow [0, 1]$ the probability of taking action a given state s .
- For a given policy, we can define how good it is via the **state-value function**: $V_\pi(s) = \mathbb{E}(\sum_{k=0} \gamma^k R_{t+1+k} \mid S_t = s)$
- $V_\pi(s)$ denotes the expected **return** if starting at s then following policy π for remainder of timesteps.
- We can write the state-value function recursively: $V_\pi(s) = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s]$
- Another related function is the **quality-function** $Q(s, a)$ that takes in state and action:

$$Q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s, A_t = a]$$

- For the optimal policy π^* , we have the following:

$$\forall s \in \mathcal{S}. \quad V^*(s) = \max_{\pi} V_\pi(s) \qquad \forall s \in \mathcal{S}. \quad \pi^* = \arg \max_{\pi} V_\pi(s)$$

$$\forall s \in \mathcal{S}, a \in \mathcal{A}. \quad Q^*(s, a) = \max_{\pi} Q_\pi(s, a)$$

$$\forall s \in \mathcal{S}. \quad V^*(s) = \max_{\pi} V_{\pi}(s)$$

- The optimal policy **adheres** to the **Bellman optimality** equations:

$$\forall s \in \mathcal{S}, a \in \mathcal{A}. \quad Q^*(s, a) = \max_{\pi} Q_{\pi}(s, a)$$

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

$$\begin{aligned} q_*(s, a) &= \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a\right] \end{aligned}$$

- This means that if we find such functions V^* or Q^* that adhere to these equations, we can construct the optimal policy as follows:

$$\begin{aligned} \pi^*(s) &= \operatorname{argmax}_{a \in A} q_*(s, a) \\ &= \operatorname{argmax}_{a \in A} \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

actions = {

1. right 

2. left 

3. up 

4. down 

}

states

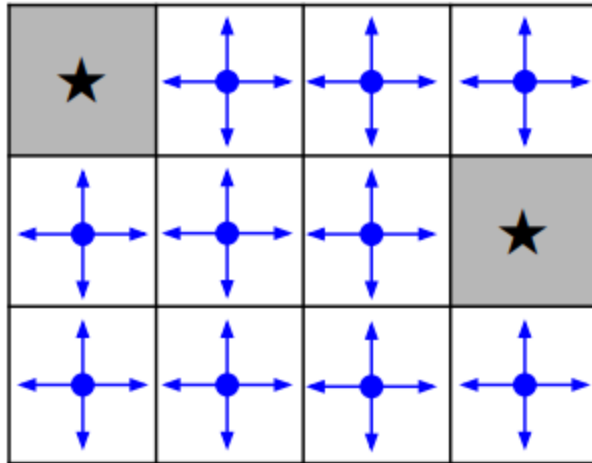
★			
			★

Set a negative “reward”
for each transition
(e.g. $r = -1$)

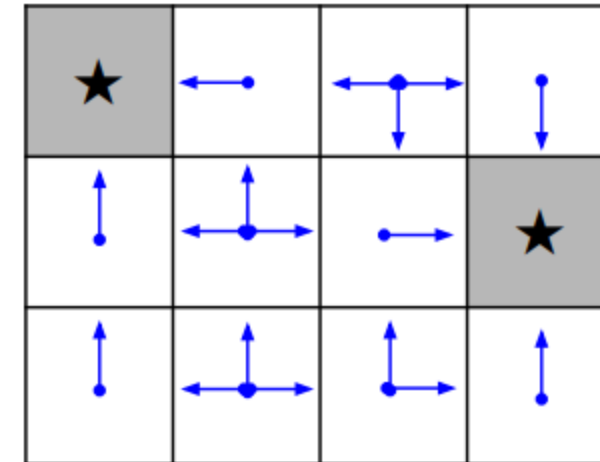
Objective: reach one of terminal states (greyed out) in
least number of actions

From [link](#)

Problem Overview – Example [2]



Random Policy

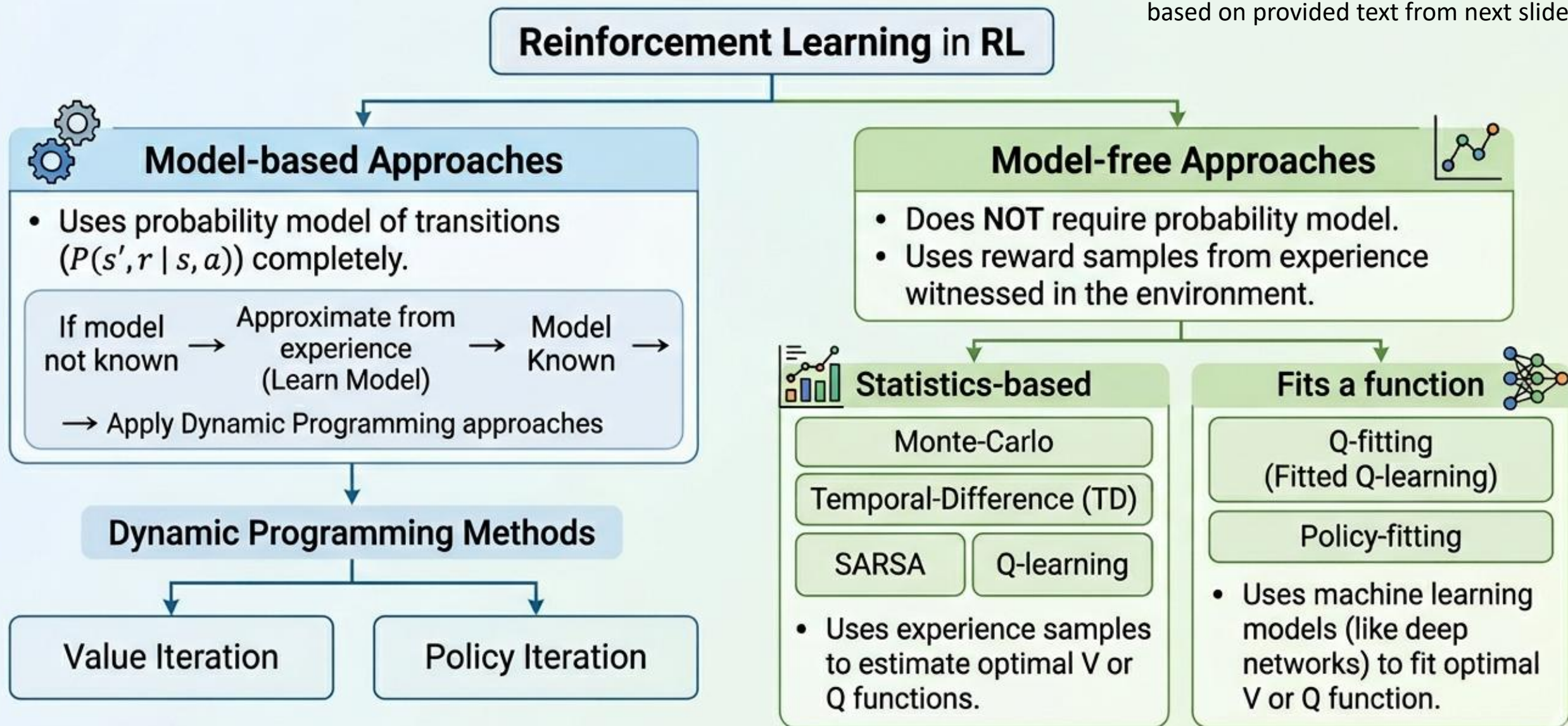


Optimal Policy

From [link](#)

Finding Optimal Policy – Options

- There are many approaches, with new approaches being proposed. We will focus on popular/fundamentals
- Model-based:
 - Called model-based because it uses the probability model of transitions, actions, and rewards completely. This is the $P(s', r \mid s, a)$ model.
 - If transition probabilities are not known, then option is to approximate it from experience (i.e. learn the model). The more experience, the better the probability approximations.
 - Once model is known, can use dynamic programming type approaches to compute the recursive equations.
 - Two popular methods: Value Iteration and Policy Iteration.
- Model-free:
 - Does not require the probability model.
 - Uses reward samples from experience witnessed through operating in the environment.
 - One approach is to focus on learning optimal V or Q functions, or learn policy directly:
 - Statistics-based: Monte-carlo, Temporal-difference, SARSA, Q-learning. Uses experience samples to estimate optimal V or Q function.
 - Fits a function: Q-fitting (also known as fitted Q-learning), policy-fitting. Uses machine learning models like deep networks to fit optimal V or Q function.
- We will discuss: value iteration, policy iteration, monte-carlo, temporal-difference, SARSA, Q-learning, fitting value and policy.



***Note:** Focuses on popular/fundamental approaches; many new methods are continuously proposed.

Finding Optimal Policy – Options

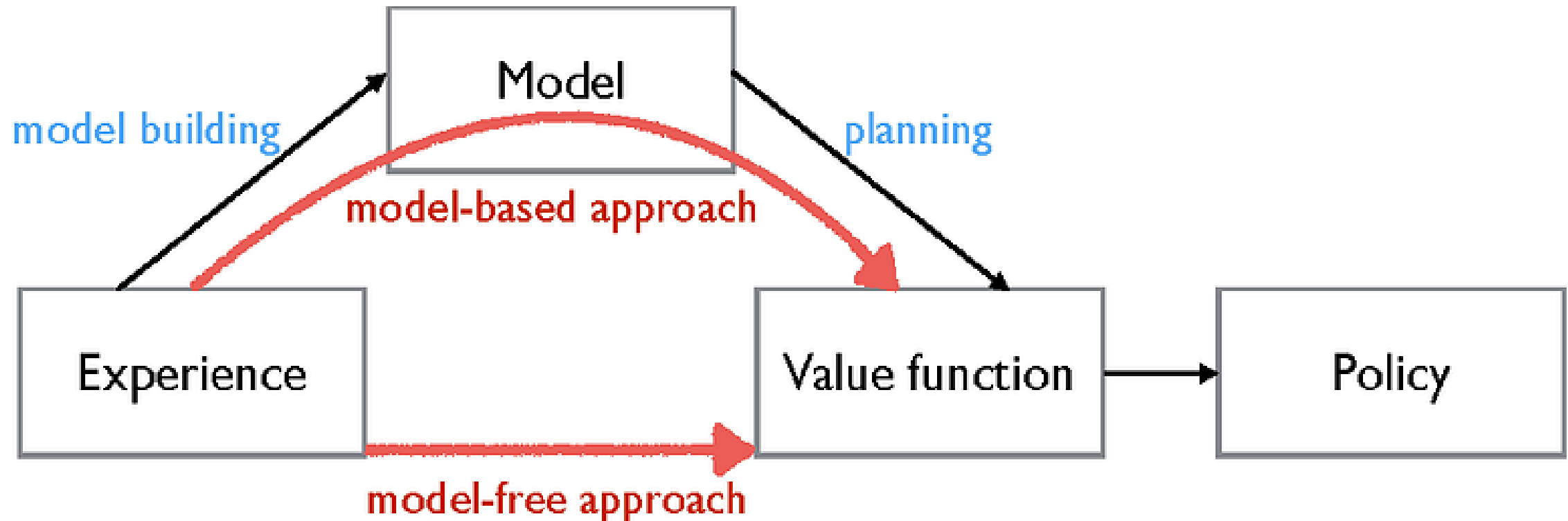


Image from [link](#)

Model-based Approaches

Recall Bellman Opt. Eqns

- The optimal policy **adheres** to the **Bellman optimality** equations:

$$\begin{aligned}v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\&= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]\end{aligned}$$

$$\begin{aligned}q_*(s, a) &= \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\&= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a\right]\end{aligned}$$

- This means that if we find such functions V^* or Q^* that adhere to these equations, we can construct the optimal policy as follows:

$$\begin{aligned}\pi^*(s) &= \operatorname{argmax}_{a \in A} q_*(s, a) \\&= \operatorname{argmax}_{a \in A} \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]\end{aligned}$$

Policy Iteration

- This is an iterative procedure, that starts with some random policy π and randomly initialized $V_0(s)$ for all states, then in each iteration:

- Policy Evaluation: evaluate the current policy π on all states via:
$$v_{k+1}(s) \doteq \mathbb{E}_{\pi}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s]$$
$$= \sum_a \pi(a|s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')]$$

Note that probability model for $p(s', r \mid s, a)$ is known.
This needs to be done multiple times for all states until the state-values no longer change (stabilizes).

This means that we have computed $V_{\pi}(s)$ for all states.

- Policy Improvement: using the current $V_{\pi}(s)$, compute a better policy via:
$$\pi'(s) \doteq \operatorname{argmax}_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi}(s')]$$

- Each iteration gets closer to the optimal policy is reached.

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

Policy Iteration – Simple Example [1]

- Consider 1x4 grid with four states A, B, C, G. The agent starts in A.
- Goal: G (terminal, reward 0). Rewards: -1 for every non-terminal step
- Actions: Left, Right. 60% success, 40% stay in place.
- Gamma: 0.9

A	B	C	G
----------	----------	----------	----------

State	Action	$p(s', r \mid s, a)$
A	Right	0.6 (B); 0.4 (A)
A	Left	0.6 (A); 0.4 (A)
B	Right	0.6 (C); 0.4 (B)
B	Left	0.6 (A); 0.4 (B)
C	Right	0.6 (G); 0.4 (C)
C	Left	0.6 (B); 0.4 (C)

- Initialize $V_c(s)$ to 0.0. Start with any policy π_c .

V_c	
$V(A)$	0.0
$V(B)$	0.0
$V(C)$	0.0

π_c	
$\pi(A)$	→
$\pi(B)$	→
$\pi(C)$	→

Note V_c at terminal states we fix to 0 by convention.

So, $V_c(G) = 0$ here.

Policy Iteration – Simple Example [2]

A	B	C	G
---	---	---	---

State	Action	p(s', r s, a)
A	Right	0.6 (B); 0.4 (A)
A	Left	0.6 (A); 0.4 (A)
B	Right	0.6 (C); 0.4 (B)
B	Left	0.6 (A); 0.4 (B)
C	Right	0.6 (G); 0.4 (C)
C	Left	0.6 (B); 0.4 (C)

Note $V_c(\text{terminal})$ we fix to 0 by convention.

Do policy evaluation

$$V_c$$

$V(A)$	0.0
$V(B)$	0.0
$V(C)$	0.0

$$\pi_c$$

$\pi(A)$	→
$\pi(B)$	→
$\pi(C)$	→

$$v_{k+1}(s) \doteq \mathbb{E}_{\pi}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s]$$

$$= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_k(s')]$$

$$V_c(A) = \pi(\rightarrow | A)(p(B, -1 | A, \rightarrow)[-1 + 0.9 * V_c(B)] + p(A, -1 | A, \rightarrow)[-1 + 0.9 * V_c(A)]) + \pi(\leftarrow | A)(p(A, -1 | A, \rightarrow)[-1 + 0.9 * V_c(A)] + p(A, -1 | A, \rightarrow)[-1 + 0.9 * V_c(A)]) = -1$$

Similarly update $V_c(B)$, $V_c(C)$;
This would update your V_c table.
Note: here we use the *old* V_c values when updating; e.g. $V_c(B)$ update uses old $V_c(A)$ value 0 not -1.

$$V_c$$

$V(A)$	-1
$V(B)$	-1
$V(C)$	-1

But you are not done policy evaluation step.

You then do **another** round with current table to update $V_c(A)$, $V_c(B)$, $V_c(C)$.

Then do another round, and so on.

You stop when the table values no longer change (stabilize).

Then you move on to policy improvement.

Policy Iteration – Simple Example [3]

Do policy evaluation

V_c			V_c			V_c			V_c			V_c	
$V(A)$	0.0	→	$V(A)$	-1	→	$V(A)$	-1.9	→	$V(A)$	-2.7100	→ ... →	$V(A)$	-3.9932
$V(B)$	0.0		$V(B)$	-1		$V(B)$	-1.9		$V(B)$	-2.4184		$V(B)$	-2.8809
$V(C)$	0.0		$V(C)$	-1		$V(C)$	-1.36		$V(C)$	-1.4896		$V(C)$	-1.5625

You stop when the table values no longer change (stabilize). Note this is **ONE** iteration of policy evaluation.

Then you move on to policy improvement.

$$\pi'(s) \doteq \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma v_\pi(s')]$$

π_c			π_c	
$\pi(A)$	→	→	$\pi(A)$	→
$\pi(B)$	→		$\pi(B)$	→
$\pi(C)$	→		$\pi(C)$	→

For (A, Right): $p(B, -1|A, \rightarrow)[-1 + 0.9 * V_c(B)] + p(A, -1|A, \rightarrow)[-1 + 0.9 * V_c(A)] = -3.9932$

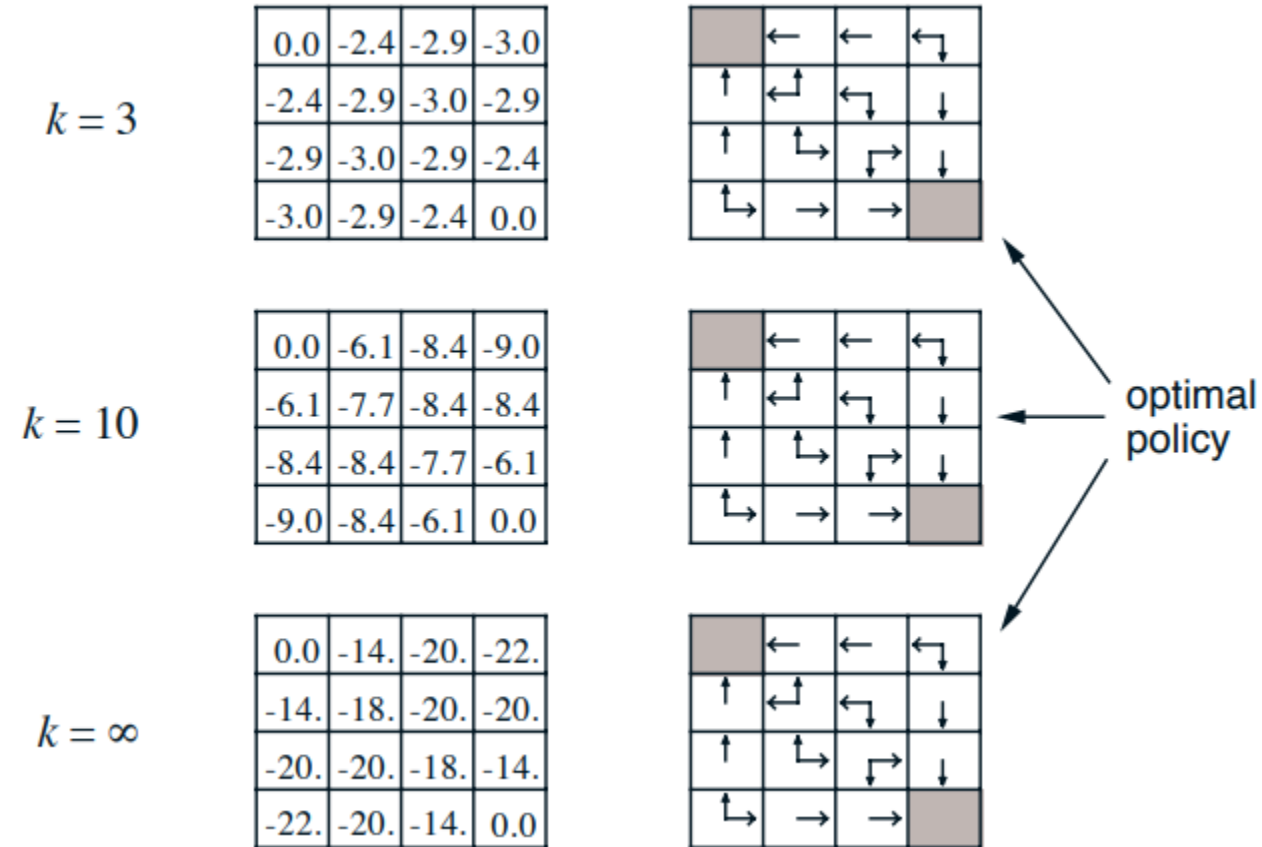
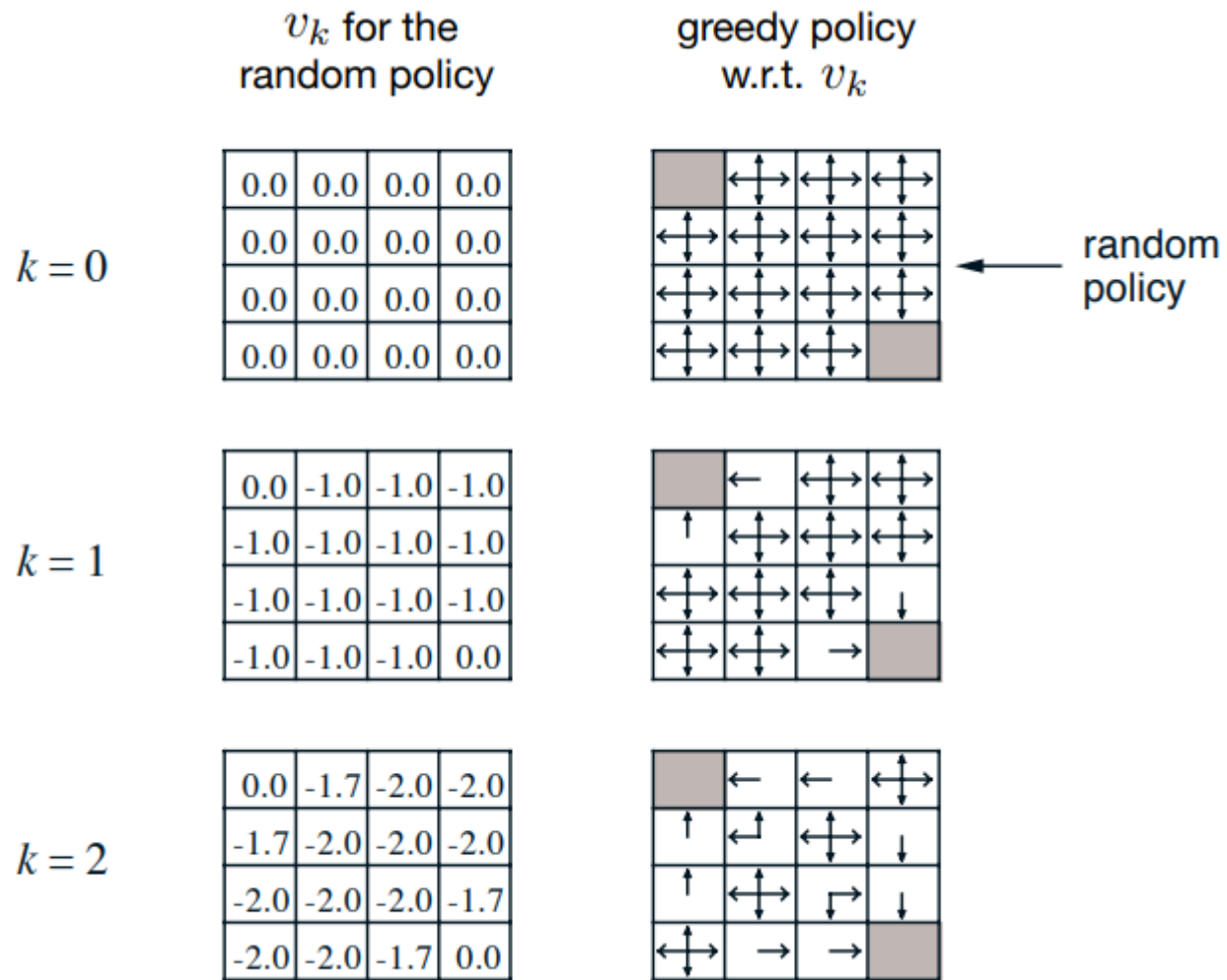
For (A, Left): $p(A, -1|A, \rightarrow)[-1 + 0.9 * V_c(A)] + p(A, -1|A, \rightarrow)[-1 + 0.9 * V_c(A)] = -4.5939$

update $\pi_c(A)$ to be Right. Similarly do for B and C.

So, we have done one iteration of policy eval + policy improvement. You do another iteration, and another, and so on, until no more policy changes occur.

Policy Iteration – Another Example

$$\pi'(s) \doteq \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) \left[r + \gamma v_{\pi}(s') \right]$$



From [link](#)

Value Iteration

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

- This is also an iterative approach, but it combines evaluation and improvement in 1 step.
- Initialize $V_0(s)$ for all states to some random value, then iterate as follows:

$$\begin{aligned} v_{k+1}(s) &\doteq \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_k(s')], \end{aligned}$$

- This update is done for all states. Repeat this until $V(s)$ converge. This means that $V \rightarrow V^*$
- Note that this formula is based on the Bellman opt. eqn. Once it converges, we can extract the optimal policy:

$$\begin{aligned} \pi^*(s) &= \operatorname{argmax}_{a \in A} q_*(s, a) \\ &= \operatorname{argmax}_{a \in A} \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

Policy vs Value Iteration Summary

- Both policy iteration and value iteration are iterative.
- Both require knowledge about the **probability model** of transitions, rewards, and actions.
- Value iteration does 1-step procedure per iteration, whereas policy iteration does 2-steps.
- Policy iteration converges faster; i.e. it requires fewer iterations.
- Issues with both methods:
 - Requires knowledge of probability model. For many problems, we do not know this.
 - Requires storing a table $V(s)$ for every state; could be problematic if state-space is exponentially large.
- Model-free approaches do not require knowledge of the probability model, we will look at the following: monte-carlo and temporal-difference (SARSA and Q-learning)
- For these model-free methods, we sometimes will operate on the Q function.

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi_*}(s, a) \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

$$\begin{aligned} q_*(s, a) &= \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a\right] \end{aligned}$$

Model-Free Approaches

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$

Loop forever (for each episode): Exploring starts ensures all (s, a) pairs are visited

Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0

Generate an episode from S_0, A_0 , following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$: Generated from current policy

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ Improve policy

Figure
from [1]

Monte-Carlo Method [2]

- Without exploring starts, can use ϵ -greedy policies to ensure all (s,a) are visited. This means with some probability, our current policy will take suboptimal action to explore.
- Issue with monte-carlo method is inefficiency. Need to run many episodes to converge to ensure sufficient (s, a) visits.

On-policy first-visit MC control (for ϵ -soft policies), estimates $\pi \approx \pi_*$

Repeat forever (for each episode):

Generate an episode following π : $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \dots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \epsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

Some probability that we sample suboptimal action (for exploration)

Figure from [1]

$$v_{\pi}(s) \doteq \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s]$$

- Instead of using cumulative rewards, we can use the recursive formula.
- Randomly initialize V or Q for all state and actions.
- Update current V or Q estimates towards a **target**: $V(S_t) \leftarrow V(S_t) + \alpha[Target - V(S_t)]$

- Simplest TD method is TD(0). It has update :

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$


TD-error. We want to shrink this difference

- Note that since TD uses *estimate* of $V(S_{t+1})$ to compute *estimate* of $V(S_t)$, we say that TD **bootstraps**. Whereas monte-carlo does **not** bootstrap.
- Advantages of TD: converges faster than monte-carlo, and can be used for non-episodic tasks.
- Two popular TD methods are: SARSA and Q-learning

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except

that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Choose A from S using policy derived from Q (e.g., ε -greedy)

Loop for each step of episode:

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

$S \leftarrow S'; A \leftarrow A';$

until S is terminal

On-policy because next action A' selected using current Q ε -greedy policy and $Q(S', A')$ used in update.

On-policy evaluation + improvement
using TD(0) update rule

Figure from [1]

On-policy vs Off-policy

“The on-policy approach in the preceding section is actually a compromise—it learns action values not for the optimal policy, but for a **near-optimal** policy that still explores.

A more straightforward approach is to use two policies, one that is learned about and that becomes the optimal policy, and one that is more exploratory and is used to generate behavior.

The policy being learned about is called the *target policy*, and the policy used to generate behavior is called the *behavior policy*. In this case we say that learning is from data “off” the target policy, and the overall process is termed off-policy learning.” — From Sutton & Barto pg. 103

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(s, a)$

Loop for each episode:

Initialize S

Loop for each step of episode:

Choose A from S using policy derived from Q (e.g., ε -greedy)

Take action A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

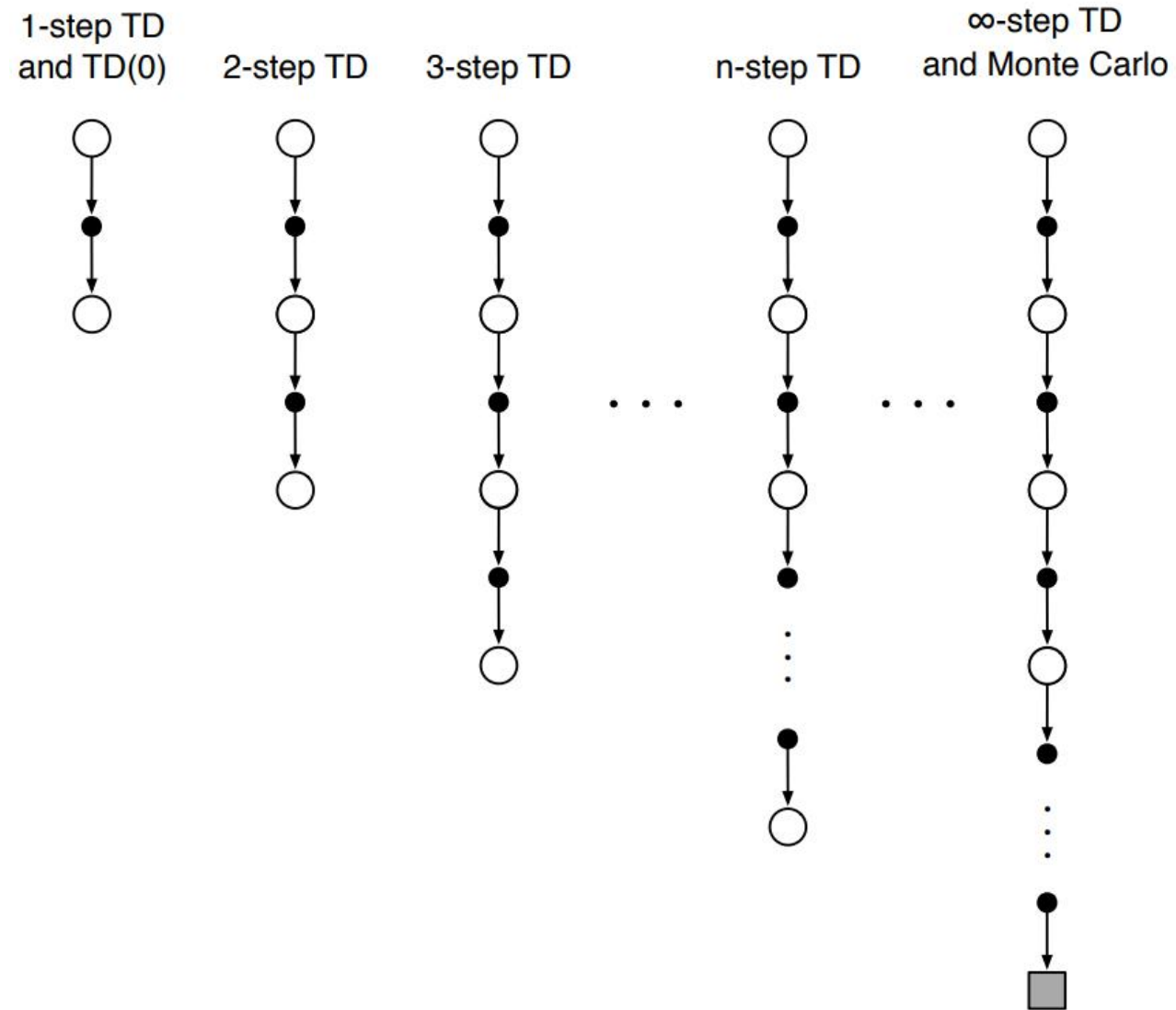
$$S \leftarrow S'$$

until S is terminal

Off-policy update because update uses greedy $\max_a Q(S', a)$ instead of using the behavior policy (ε -greedy Q)

Temporal-Difference (TD) Methods Summary

- TD-target biased (because of bootstrap estimate) but lower variance than MC-target. In practice, TD methods converge faster. From Sutton & Barto pg. 124 no proof as of yet.
- Other types of TD targets: n -step target and lambda-return targets/eligibility traces. Requires more computation than TD(0), but converges faster.
- For example, TD(1) target uses two step lookahead target: $\mathbf{r}_k + \gamma \mathbf{r}_{k+1} + \gamma^2 V(\mathbf{s}_{k+2})$
- In the limit, TD(∞) target converges to monte-carlo target (cumulative reward estimate).
- SARSA is on-policy because it uses the a_{t+1} selected by current ϵ -greedy Q policy in the update.
- Q-learning is off-policy because it uses an ϵ -greedy policy for the behavior, but uses $\max_a Q(s_{t+1}, a)$ for update. Preferred approach because we can use behavior policies that follow expert humans (like grandmaster chess).



Q-fitting using ML (deep learning)

Function Approximation of Value Functions

- Previous methods use tables for storing value functions; one entry for each state or state-action pair.
- Some tasks have a large state-space and/or action-space. Memory issue, but also experience issue. Many states/actions may never be visited.
- Solution: approximate the V or Q function via linear/non-linear function with parameters w

$$\hat{v}(s; \mathbf{w}) \approx v_*(s) \qquad \hat{q}(s, a; \mathbf{w}) \approx q_*(s, a)$$

- With gradient descent based methods, we minimize the MSE:

$$MSE(\mathbf{w}) = \sum_{s \in S} (q_*(s, a) - \hat{q}(s, a; \mathbf{w}))^2$$

- In practice, we don't know q_* , so we approximate it via a target U_t . Parameter update becomes:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [U_t - \hat{q}(S_t, A_t; \mathbf{w}_t)] \nabla \hat{q}(S_t, A_t; \mathbf{w}_t)$$

- This can be done using ML supervised learning approach.

Fitted Q-Learning (Riedmiller 2005)

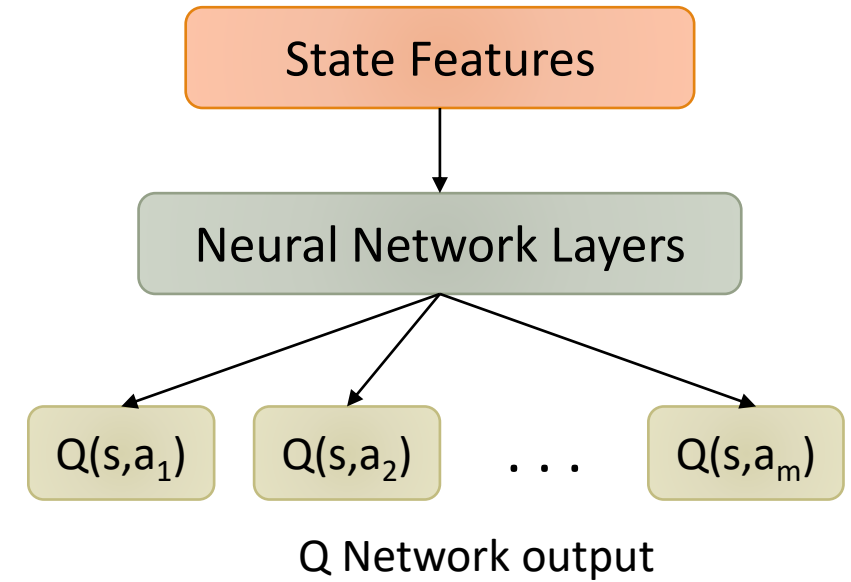
- The samples are of the form: $\langle s, a, r, s' \rangle$.
- Approximate q_* by an ML model with parameters θ
- Target label is: $Y_k = r + \gamma \max_{a' \in A} Q(s', a'; \theta_k)$
- Minimize the MSE using gradient descent:

$$L_{DQN} = (Q(s, a; \theta_k) - Y_k)^2$$

$$\theta_{k+1} = \theta_k + \alpha(Y_k - Q(s, a; \theta_k)) \nabla_{\theta_k} Q(s, a; \theta_k)$$

- Update is applied after each sample. Can lead to slow convergence or **instability**.

“One reason for this is, that if weights are adjusted for one certain state action pair, then unpredictable changes also occur at other places in the state-action space.” — Riedmiller 2005



Deep Q-Networks DQN (Mnih et al. 2015)

- Successfully applied to ATARI games. To limit instabilities, it does:

1. Network weights θ_k are updated every iteration (like previous), but the target uses an “older” network via older weights θ_k^- . Every C iterations, we update older weights to newer $\theta_k^- = \theta_k$.

$$Y_k^{DQN} = r + \gamma \max_{a' \in A} Q(s', a'; \theta_k^-)$$

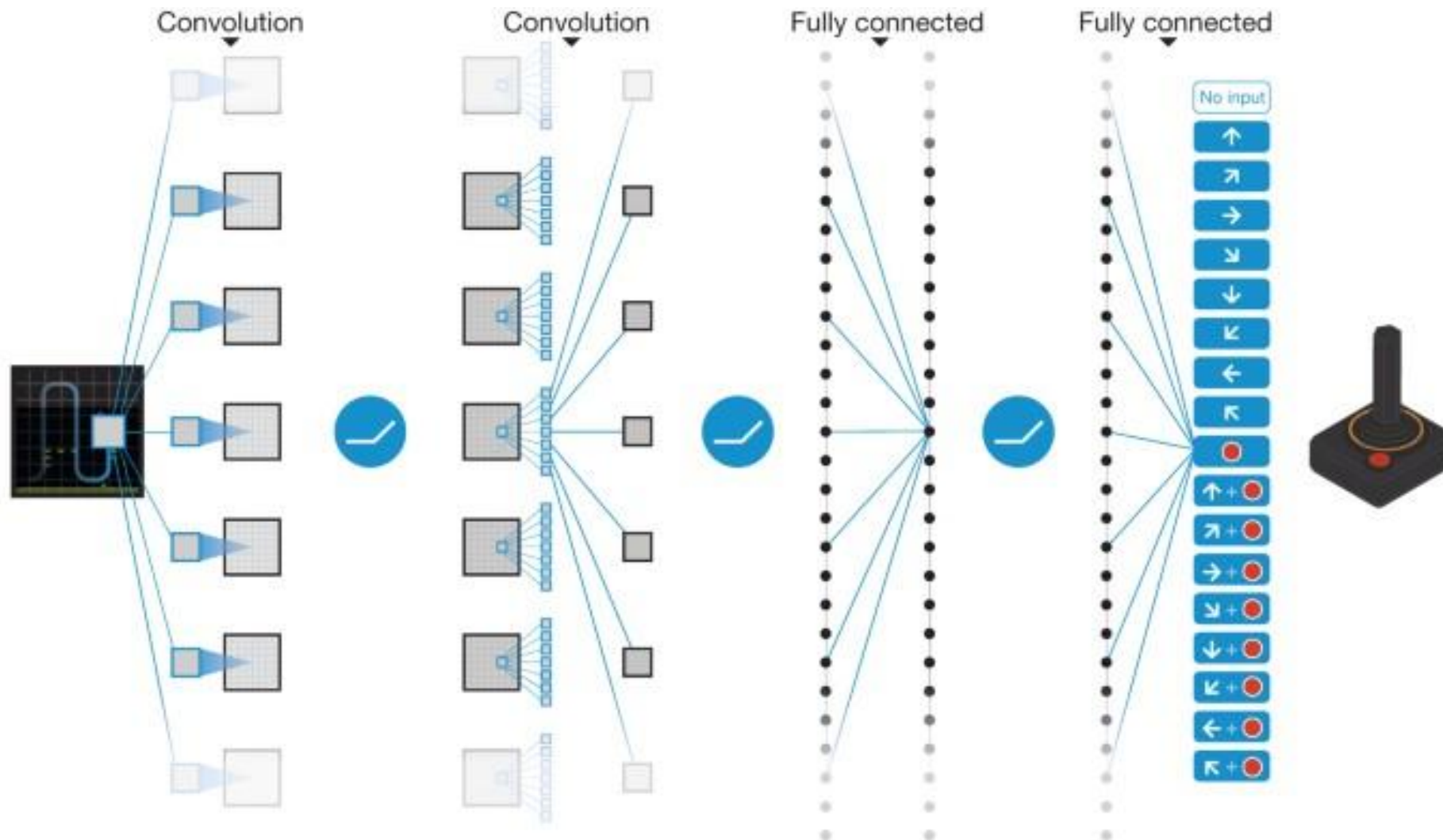
2. Clips rewards between -1 and +1. Reasoning:

“As the scale of scores varies greatly from game to game, we clipped all positive rewards at 1 and all negative rewards at -1, leaving 0 rewards unchanged. Clipping the rewards in this manner limits the scale of the error derivatives and makes it easier to use the same learning rate across multiple games.” – Mnih et. Al. 2015

- Uses deep learning techniques: image preprocessing, normalization, CNN architectures, and optimizers.
- Code review and discussion: https://keras.io/examples/rl/deep_q_network_breakout/

Deep Q-Networks DQN (Mnih et al. 2015)

- Uses deep learning techniques: image preprocessing, normalization, CNN architectures, and optimizers.



Double DQN (Van Hasselt et al. 2016)

- Problem with this target: $Y_k^{DQN} = r + \gamma \max_{a' \in A} Q(s', a'; \theta_k^-)$
- The max operation done on estimates, leading to a positive bias.
- Example: true $q(s, a) = 0$ for all a , but estimate network $\hat{q}(s, a)$ noisy about 0 -> max will be positive.
- “Overestimation combined with bootstrapping then has the pernicious effect of propagating the wrong relative information about which states are more valuable than others, directly affecting the quality of the learned policies.” – Van Hasselt et al. 2016
- Solution: **decouple** the **selection** of the maximum action and the **estimation** of the value of the maximum action.

$$Y_k^{DDQN} = r + \gamma \underbrace{Q(s', \overbrace{\operatorname{argmax}_{a \in A} Q(s', a; \theta_k)}^{\text{Selection of maximum action using current weights}}); \theta_k^-}_{\text{Estimation of selected maximum using old weights}}$$

- “The idea of Double Q-learning is to reduce overestimations by decomposing the max operation in the target into action selection and action evaluation.” – Van Hasselt et al. 2016

- Makes use of Advantage function: $A(s, a)$. How much better is it to perform action a in state s versus following the current policy?
- Dueling Network uses same target as DQN: $Y_k = r + \gamma \max_{a' \in A} Q(s', a'; \theta^{(1)}, \theta^{(2)}, \theta^{(3)})$
- Writes Q as:

$$Q(s, a; \theta^{(1)}, \theta^{(2)}, \theta^{(3)}) = V(s; \theta^{(1)}, \theta^{(3)}) + \underbrace{\left(A(s, a; \theta^{(1)}, \theta^{(2)}) - \max_{a' \in \mathcal{A}} A(s, a'; \theta^{(1)}, \theta^{(2)}) \right)}_{\text{Get closer to maximum advantage action. i.e. get this difference to 0.}}$$

Get closer to maximum advantage action.
i.e. get this difference to 0.

Dueling Network Architecture [2] (Wang et al. 2015)

$$Q(s, a; \theta^{(1)}, \theta^{(2)}, \theta^{(3)}) = V(s; \theta^{(1)}, \theta^{(3)}) + \left(A(s, a; \theta^{(1)}, \theta^{(2)}) - \max_{a' \in \mathcal{A}} A(s, a'; \theta^{(1)}, \theta^{(2)}) \right)$$

- Dueling network can be thought of as three parts:
 - Beginning of network θ_1 focuses on learning from state features.
 - Advantage-network θ_2 learn the advantage.
 - V-network θ_3 to learn the optimal V function.

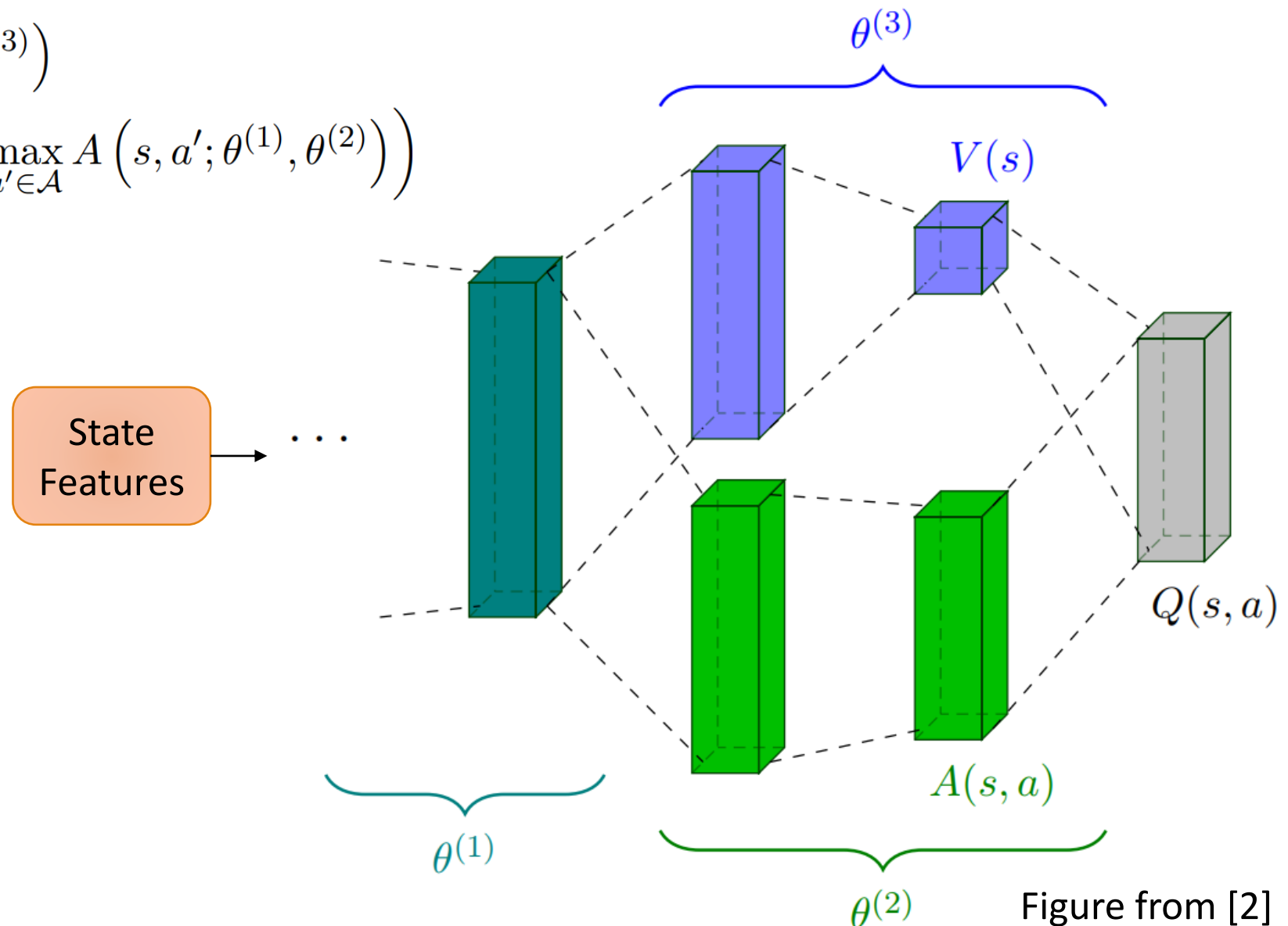


Figure from [2]

- For stability, uses this in practice:

$$Q(s, a; \theta^{(1)}, \theta^{(2)}, \theta^{(3)}) = V(s; \theta^{(1)}, \theta^{(3)}) + \underbrace{\left(A(s, a; \theta^{(1)}, \theta^{(2)}) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a'; \theta^{(1)}, \theta^{(2)}) \right)}$$

Get closer to mean advantage action

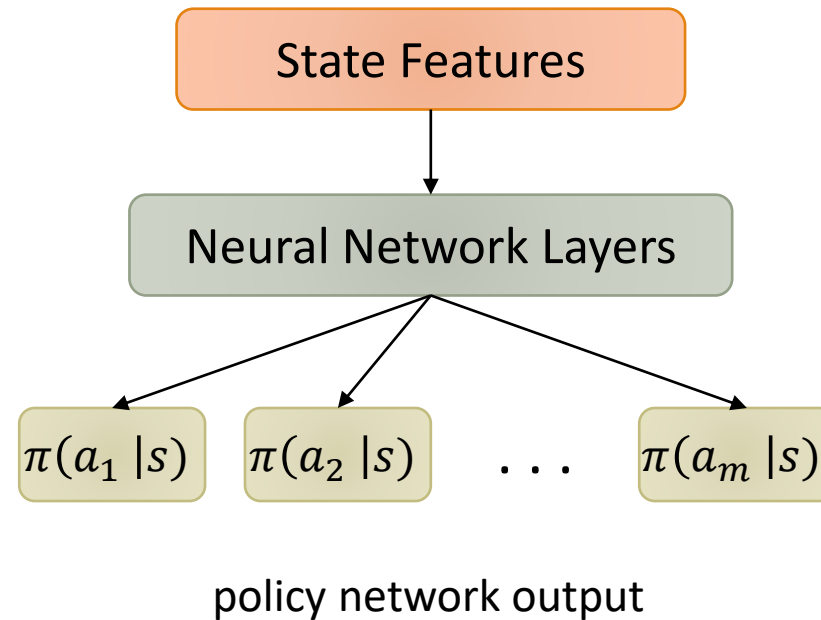
Summary of value function approximation with ML [1]

- We can use an ML model to learn the optimal value functions. Here we showed work done with deep learning models for Q-learning (DQNs), but technically you could use other ML models.
- The techniques we just showed build upon each other to get deep learning to train correctly. This involved reward shaping, modifying the target label, modifying training frequency, modifying network architecture, etc.
- Hessel et al., 2017 achieved state-of-the-art performance with DQNs on ATARI benchmarks by combining techniques: Double DQN, replay memory (prioritized), Dueling networks, reward shaping, etc.
- Relatively simple to implement with open-source frameworks: OpenAI Gym, Keras, Tensorflow, pytorch, KerasRL.
- Note that training a model involves playing through games collecting experience (samples $\langle s, a, r, s' \rangle$). These samples are used to train the network (modify the weights to approximate Q).
- One nice thing about this is that the policy used to play the game does not need to be the network policy (off-policy). So, we can use expert games to train the model, as well as randomized policy (for exploration).
- These network architectures can be customized. The input to the network can be rich such as containing all data (states, actions, rewards) since the start of the game.
- See more Q-networks: <https://paperswithcode.com/methods/category/q-learning>

Policy-learning using ML (deep learning)

Policy Gradient Methods [1]

- Model the policy with a parametrized function (i.e. a deep network): $\pi_{\theta}(a|s)$



- Note that each output is the probability of the action given the input. Once trained correctly, it would learn optimal policy, and so select action with highest probability.

Policy Gradient Methods [1]

- Model the policy with a parametrized function (i.e. a deep network): $\pi_{\theta}(a|s)$
- Find a policy that maximizes the value function: $J(\theta) = v_{\pi_{\theta}}(s_0)$
$$= \sum_a \pi_{\theta}(a|s_0) \sum_{s', r} p(s', r|s_0, a) [r + \gamma v_{\pi_{\theta}}(s')]$$
$$= \sum_a \pi_{\theta}(a|s_0) q_{\pi_{\theta}}(s_0, a)$$
- Policy Gradient Theorem:

$$\begin{aligned}\nabla_{\theta} J(\theta) &\propto \sum_s \mu(s) \sum_a q_{\pi_{\theta}}(s, a) \nabla_{\theta} \pi_{\theta}(a|s) \\ &= \sum_s \mu(s) \sum_a q_{\pi_{\theta}}(s, a) \pi_{\theta}(a|s) \frac{\nabla_{\theta} \pi_{\theta}(a|s)}{\pi_{\theta}(a|s)} \\ &= \mathbb{E}_{s \sim \mu, a \sim \pi_{\theta}} [q_{\pi_{\theta}}(s, a) \nabla_{\theta} \ln(\pi_{\theta}(a|s))]\end{aligned}$$

- This tells us that the gradient of the objective function can be estimated from samples. We just need to be able to compute $q_{\pi_{\theta}}(s, a)$ and $\nabla_{\theta} \ln \pi_{\theta}(a|s)$. Then parameter update is:
$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta_t} J(\theta_t)$$

Actor-Critic Method [1]

- The policy gradient theorem tells us that the gradient of the objective function can be estimated from samples. We just need to be able to compute $q_{\pi_{\theta}}(s, a)$ and $\nabla_{\theta} \ln \pi_{\theta}(a|s)$.
- There are options on how to learn parameters of the optimal policy, we will look at *one* popular method: **actor-critic**.
- Actor-critic learns both the Q function and the policy.
- The actor is a deep neural network representing the policy $\pi(s, a; \theta)$
- The critic is deep neural network representing Q function (like deep Q-networks we have seen) $q(s, a; \theta_2)$.
- When training on game samples, both networks are updated:

Update DQN network: $\theta_{k+1} = \theta_k + \alpha(Y_k - Q(s, a; \theta_k)) \nabla_{\theta_k} Q(s, a; \theta_k)$

Update policy network: $\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} ((\log \pi(s_k, a_k, \theta)) Q(s_k, a_k, \theta_2))$

Notice that policy network update uses Q-network

Model Free – Policy Gradient Methods – Actor-Critic

- Actor-critic learns both the Q function and the policy.
- The **actor** is a deep network representing the policy $\pi(s, a; \theta)$
- The **critic** is deep network representing Q function (like deep Q-networks we have seen) $q(s, a; \theta_2)$.
- Both networks updated together.
- See more at:
<https://paperswithcode.com/methods/category/policy-gradient-methods>

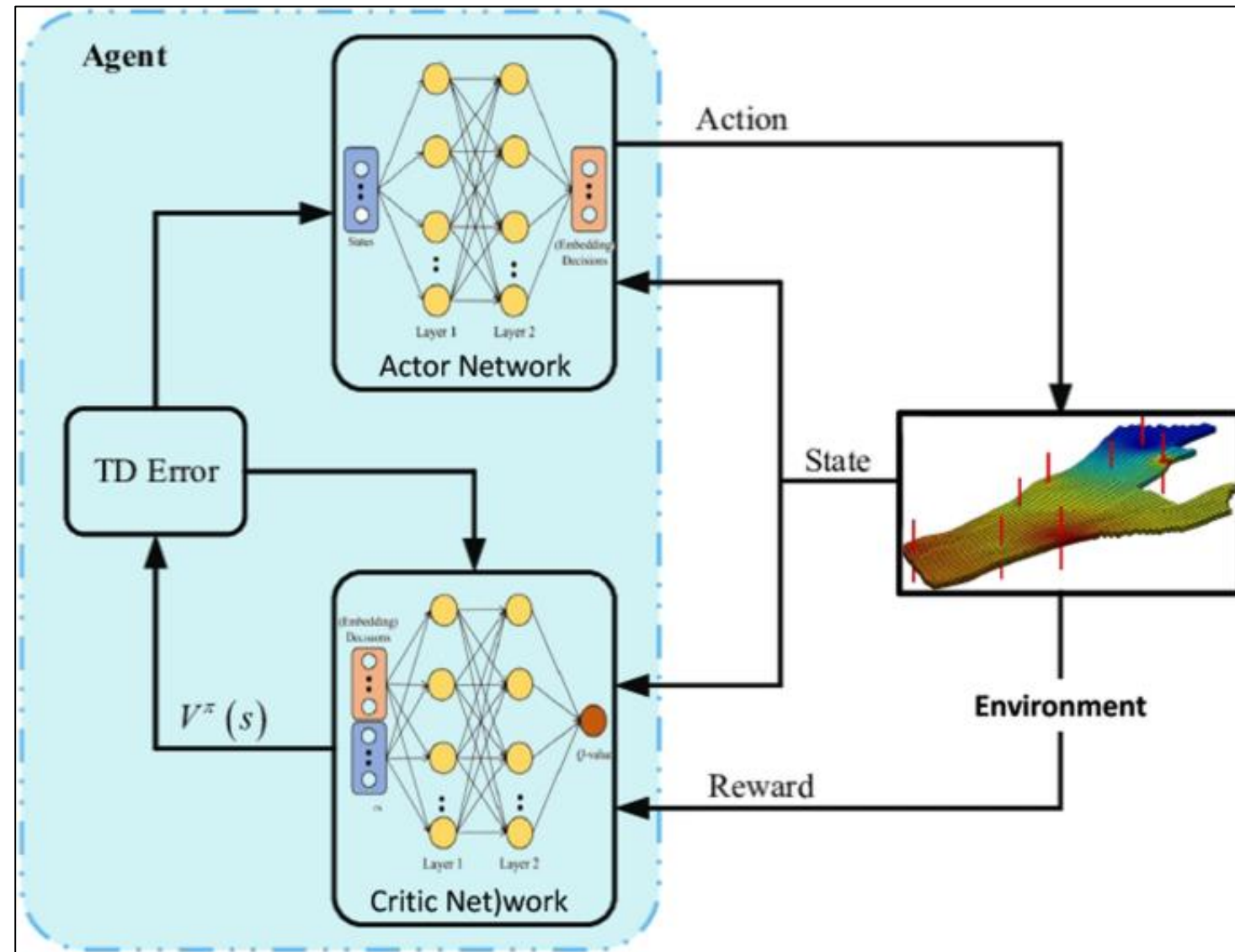


Figure from [link](#)

Actor-Critic Method [2]

- Recent work on actor-critic methods have proposed other update rules for stable training. We will mention them here but won't go into details.
- Trust Region Optimization (TRPO) [Schulman et. al. 2015]
- Proximal Policy Optimization (PPO) [Schulman et. al. 2017]. Simpler formula than TRPO, and gives similar performance. Easy to implement. Uses a single network with shared parameters to output Q and π .
- Deterministic policy methods like Deterministic policy gradient (DPG) [Silver et. al. 2014] and Deep DPG [Lillicrap et. al. 2015]. Sutton and Barto book state that stochastic policies will approach deterministic policies (if they are optimal) asymptotically [pg. 322].
- See more at: <https://paperswithcode.com/methods/category/policy-gradient-methods>
- Code review and discussion: https://keras.io/examples/rl/ppo_cartpole/

Frameworks and Milestones

Interesting RL frameworks

- [gym](#). Python library containing a wide range of RL environments. Good for learning, testing, and developing RL strategies.
- [KerasRL](#): deep learning RL package. Has value-based and policy-based deep learning methods.
- [AirSim](#): simulator for cars, planes, drones in UnrealEngine.
- [Carla](#): simulator for autonomous driving research.
- See more recommended libraries at this [link](#)

Milestones

- DeepMind Human level performance on ATARI games [Mnih et al 2015]
- DeepMind's AlphaZero for Chess, Shogi and Go [Silver et al 2017]
- OpenAI's Five neural networks for playing DOTA [2019]. Uses PPO.

“The key ingredient in solving this complex environment was to scale existing reinforcement learning systems to unprecedented levels, utilizing thousands of GPUs over multiple months. We built a distributed training system to do this which we used to train a Dota 2-playing agent called OpenAI Five.” - [paper](#)

- Deepmind Starcraft II grandmaster level play [2019]. Uses modified IMPALA. [Link](#)

OpenAI's Five Infrastructure

	OPENAI 1V1 BOT	OPENAI FIVE
CPUs	60,000 CPU cores on Azure	128,000 <u>preemptible</u> CPU cores on GCP
GPUs	256 K80 GPUs on Azure	256 P100 GPUs on GCP
Experience collected	~300 years per day	~180 years per day (~900 years per day counting each hero separately)
Size of observation	~3.3 kB	~36.8 kB
Observations per second of gameplay	10	7.5
Batch size	8,388,608 observations	1,048,576 observations
Batches per minute	~20	~60

Recap of this course

Navigation & Searching

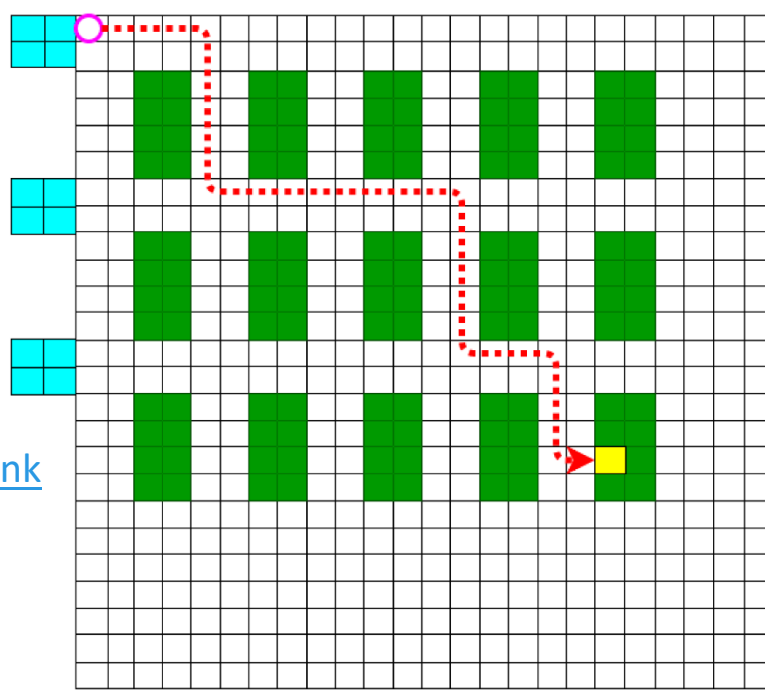


Image from: [link](#)

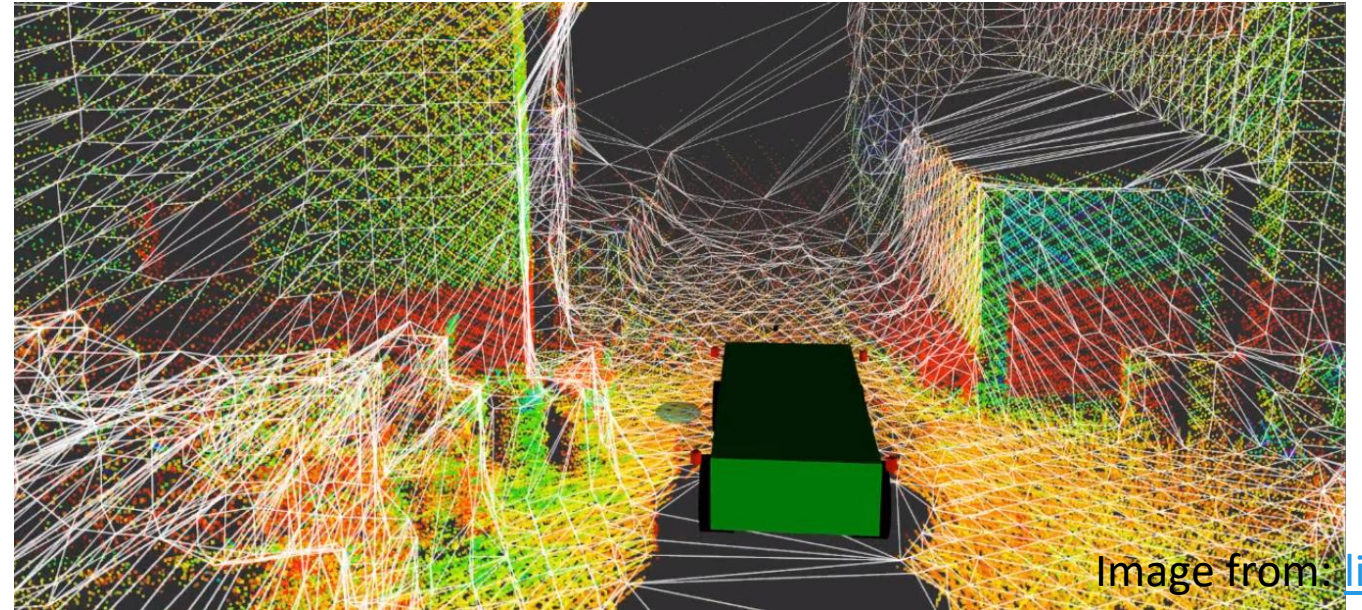
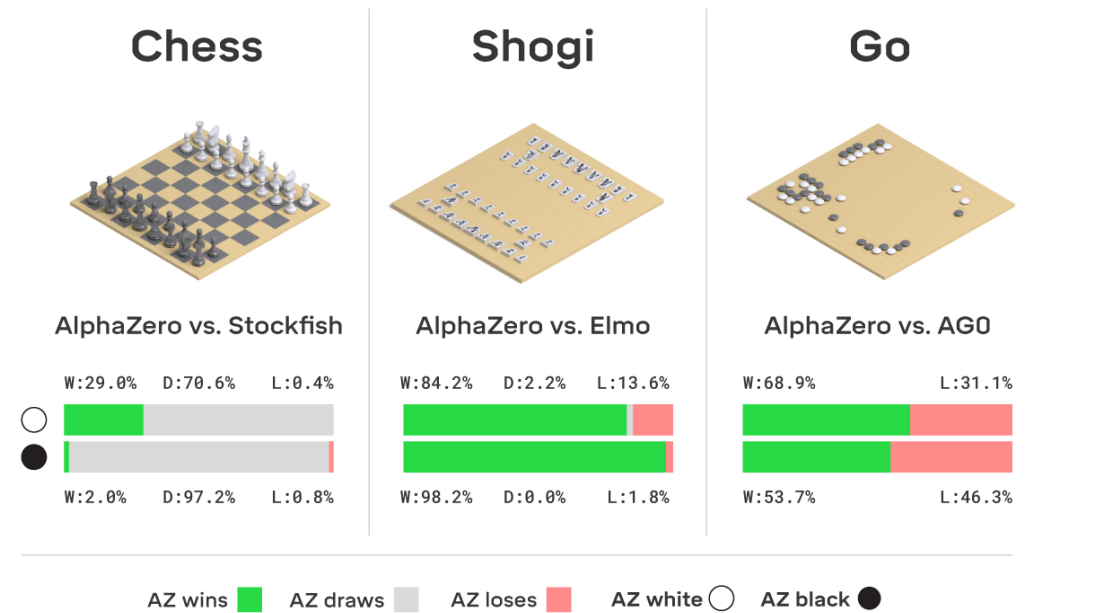


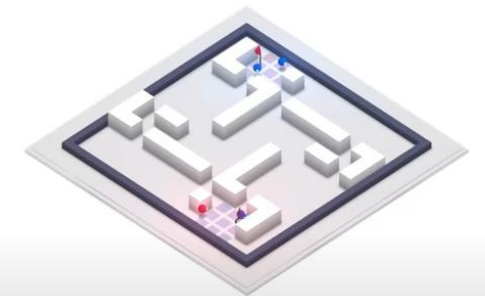
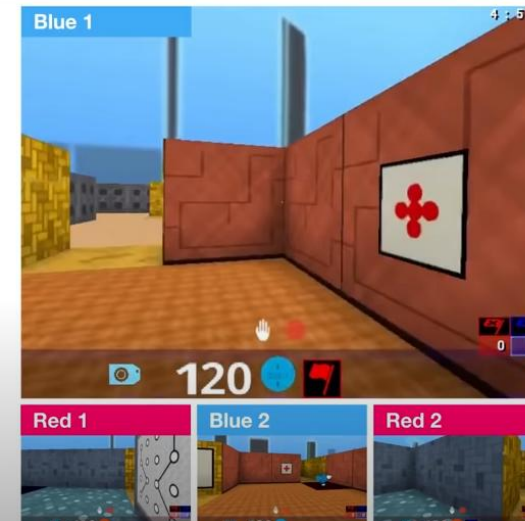
Image from: [link](#)

Game Playing

- DeepBlue defeats chess champion [1997]
- AlphaZero defeats humans and other AI in chess, shogi and go [2018].
- Open AI Five beats Dota champions. [2018]
- [Deepmind's Quake 3 AI](#) [2018].
- [AlphaStar](#): Grandmaster level in StarCraft II [2019].



Agent observation raw pixels



Scheduling/Optimizing

- Finding optimal configuration of a system.
- Job scheduling
- Circuit design
- Cargo packing
- Flight scheduling
- Registrar scheduling

Course Offering

Select Term

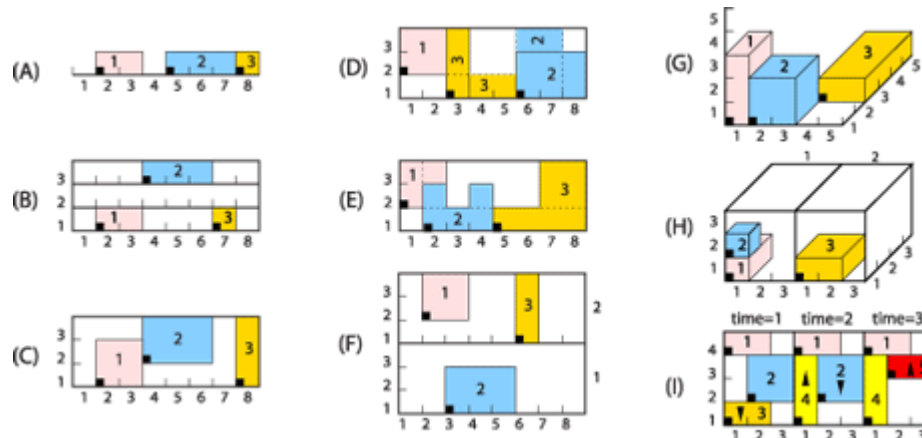
Term 222

Select Department

Accounting & Finance

Course-Sec	Activity	CRN	Course Name	Instructor	Day	Time	Location	Status	Waitlist
ACCT110-01	LEC	21684	Introduction to Financial Accounting	ABDULLAH ALHARBI	MW	1230-1345	24-174	Closed	Open
ACCT110-02	LEC	21083	Introduction to Financial Accounting	ABDULLAH ALHARBI	MW	1400-1515	24-174	Open	Closed
ACCT210-01	LEC	21087	Introduction to Managerial Accounting	MIRGHANI AHMED	UT	1100-1215	24-174	Closed	Open
ACCT210-04	LEC	21090	Introduction to Managerial Accounting	WALID CHEFFI	MW	1230-1345	24-244	Open	Closed
ACCT210-05	LEC	21982	Introduction to Managerial Accounting	MIRGHANI AHMED	UT	1400-1515	24-250	Open	Closed

Course-instructor-room scheduling



Packing problems. Image from [link](#)

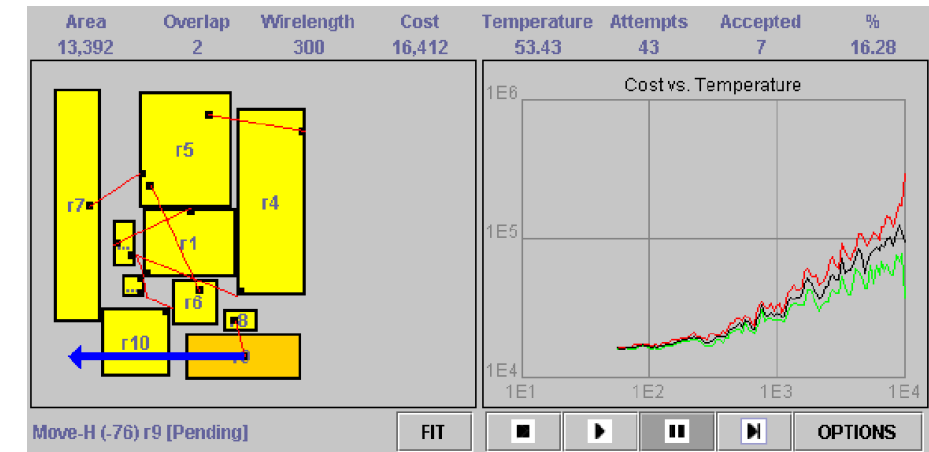


Figure 5 – Simulated Annealing Placement Applet

Circuit design. Image from [link](#)

Other Searches and Automated Planning

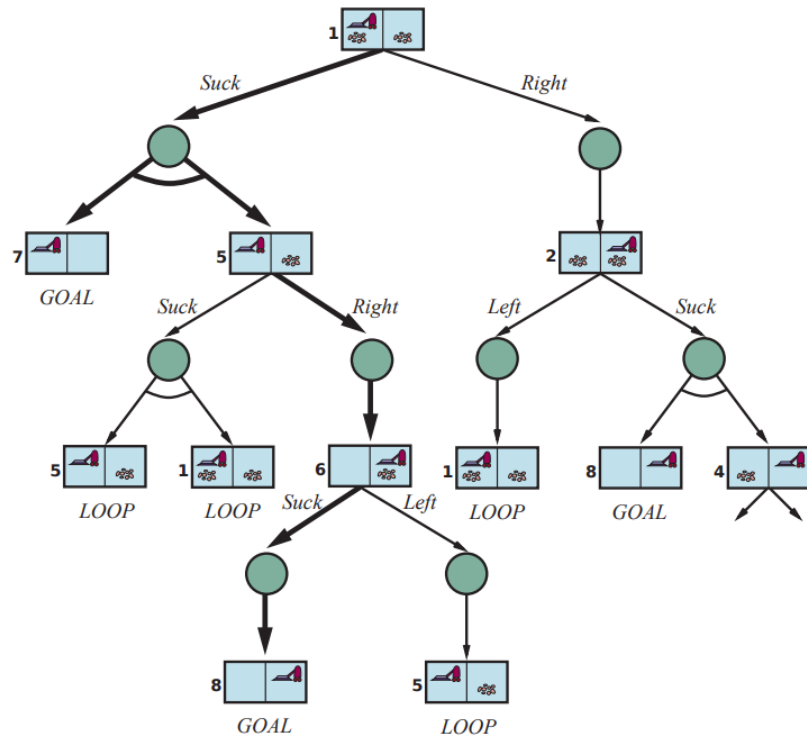
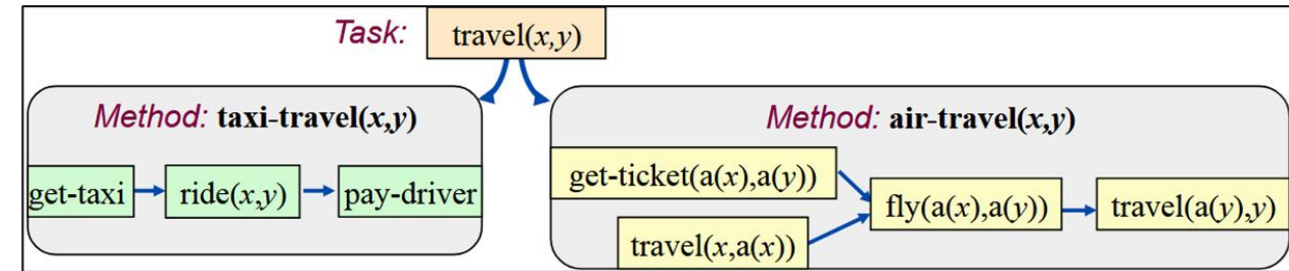


Figure 4.10 The first two levels of the search tree for the erratic vacuum world. State nodes are OR nodes where some action must be chosen. At the AND nodes, shown as circles, every outcome must be handled, as indicated by the arc linking the outgoing branches. The solution found is shown in bold lines.



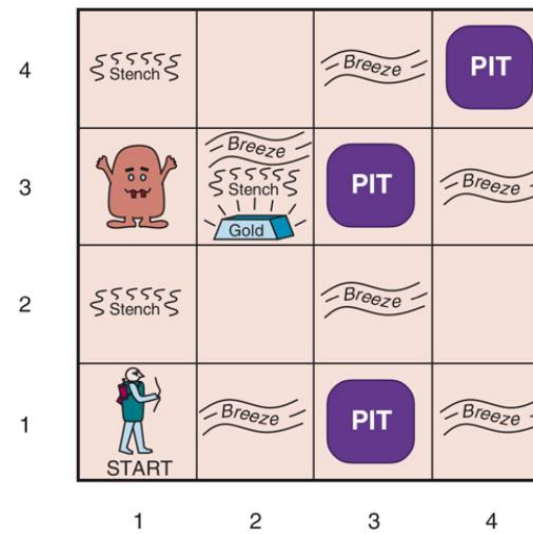
```

1 (define (domain travel-htn)
2   (:requirements :negative-preconditions :typing :hierarchy)
3   (:types
4     airport - place
5     place - object
6   )
7   (:predicates
8     (use-taxi ?from - place ?to - place)
9     (use-air ?from - place ?to - place)
10  )
11
12  (:task travel
13    :parameters (?x - place ?y - place)
14  )
15  (:task get-ticket
16    :parameters (?a1 - airport ?a2 - airport)
17  )
18

```

Decision-making/Probabilistic-Agents

- Knowledge-based agents
- Theorem-provers
- Medical diagnosis systems
- Automated trading
- Fraud Detection
- Fault Detection



1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 B OK	2,2	3,2	4,2
1,1 OK	2,1 B OK	3,1	4,1

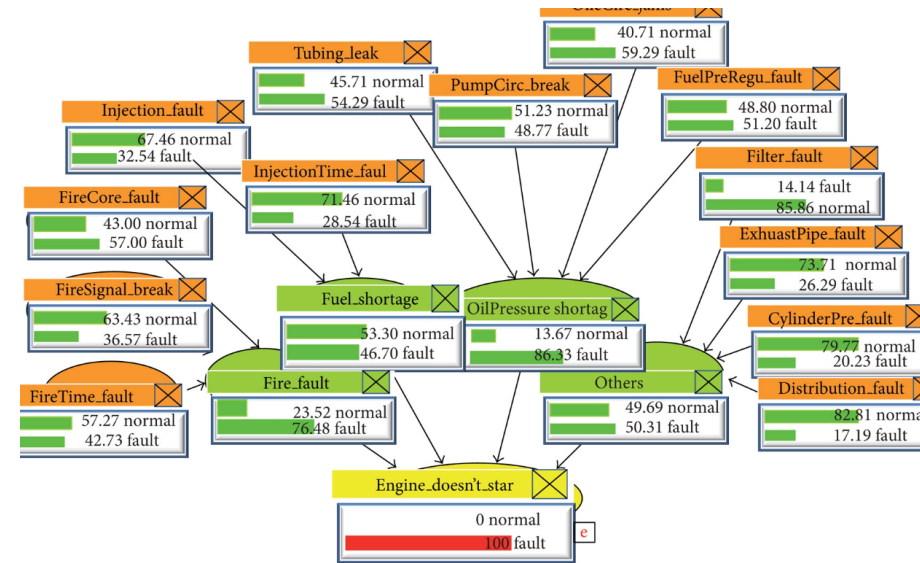
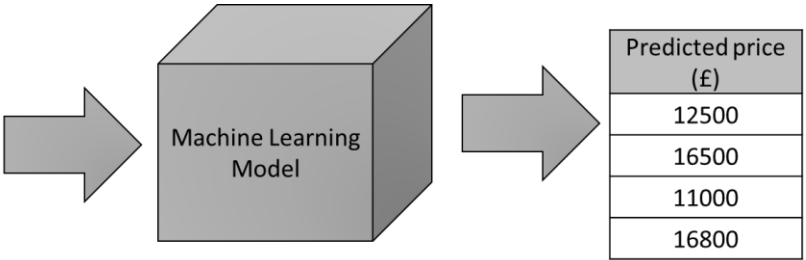


Image from [link](#)

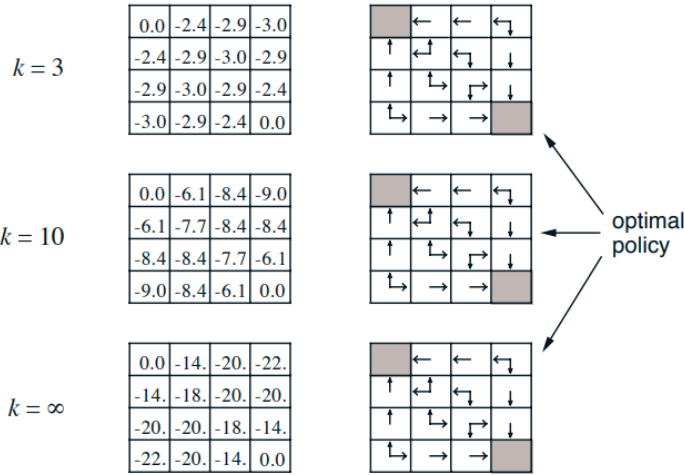
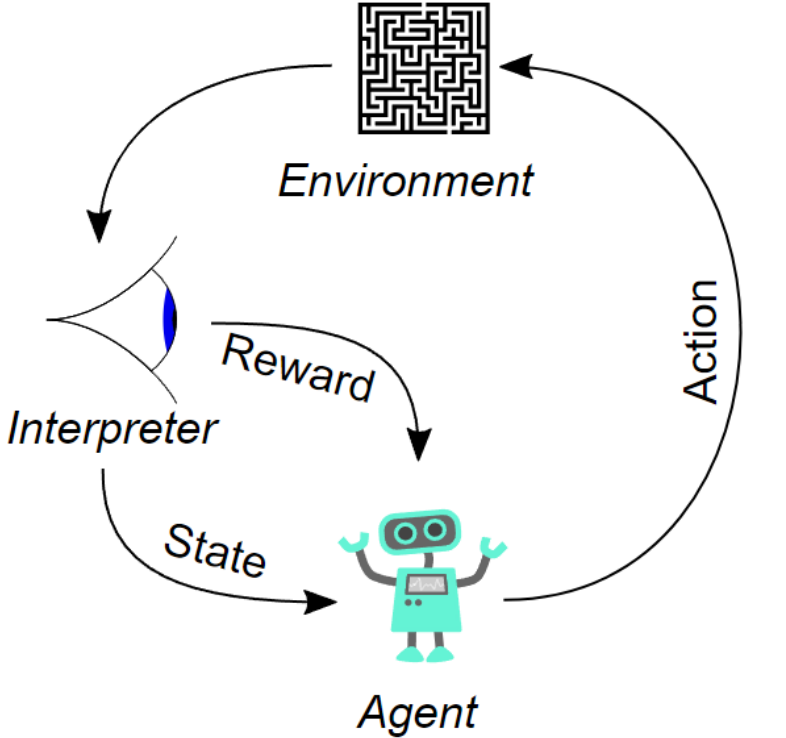
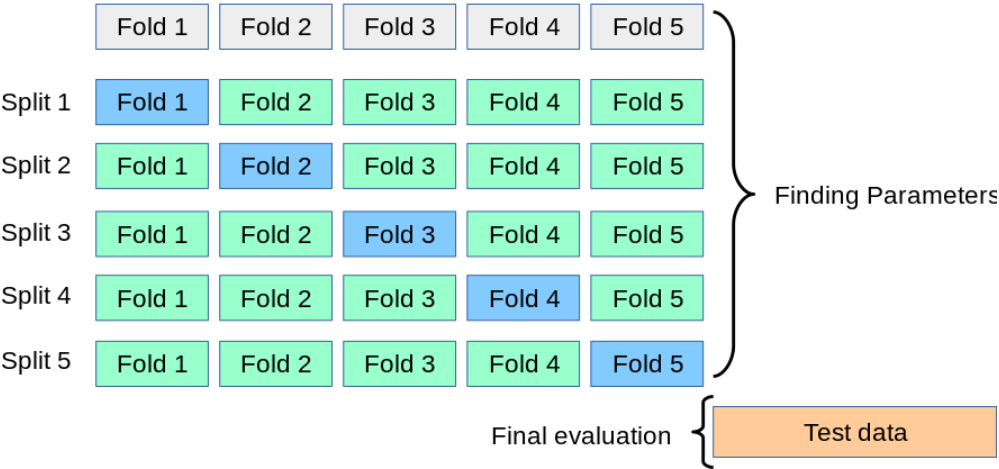
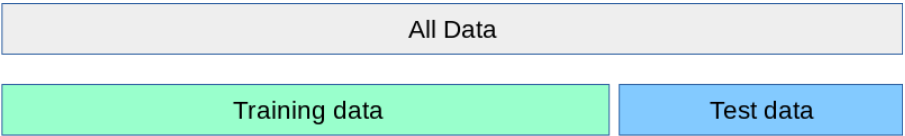
Machine Learning, RL

Model	Year	Transmission	Mileage	mpg	Engine Size
A1	2017	Manual	15735	55.4	1.4
A6	2016	Automatic	36203	64.2	2
A1	2016	Manual	29946	55.4	1.4
A4	2017	Automatic	25952	67.3	2

Input is a list of car descriptors.
Each row represents a car.



Predicted price (£)
12500
16500
11000
16800



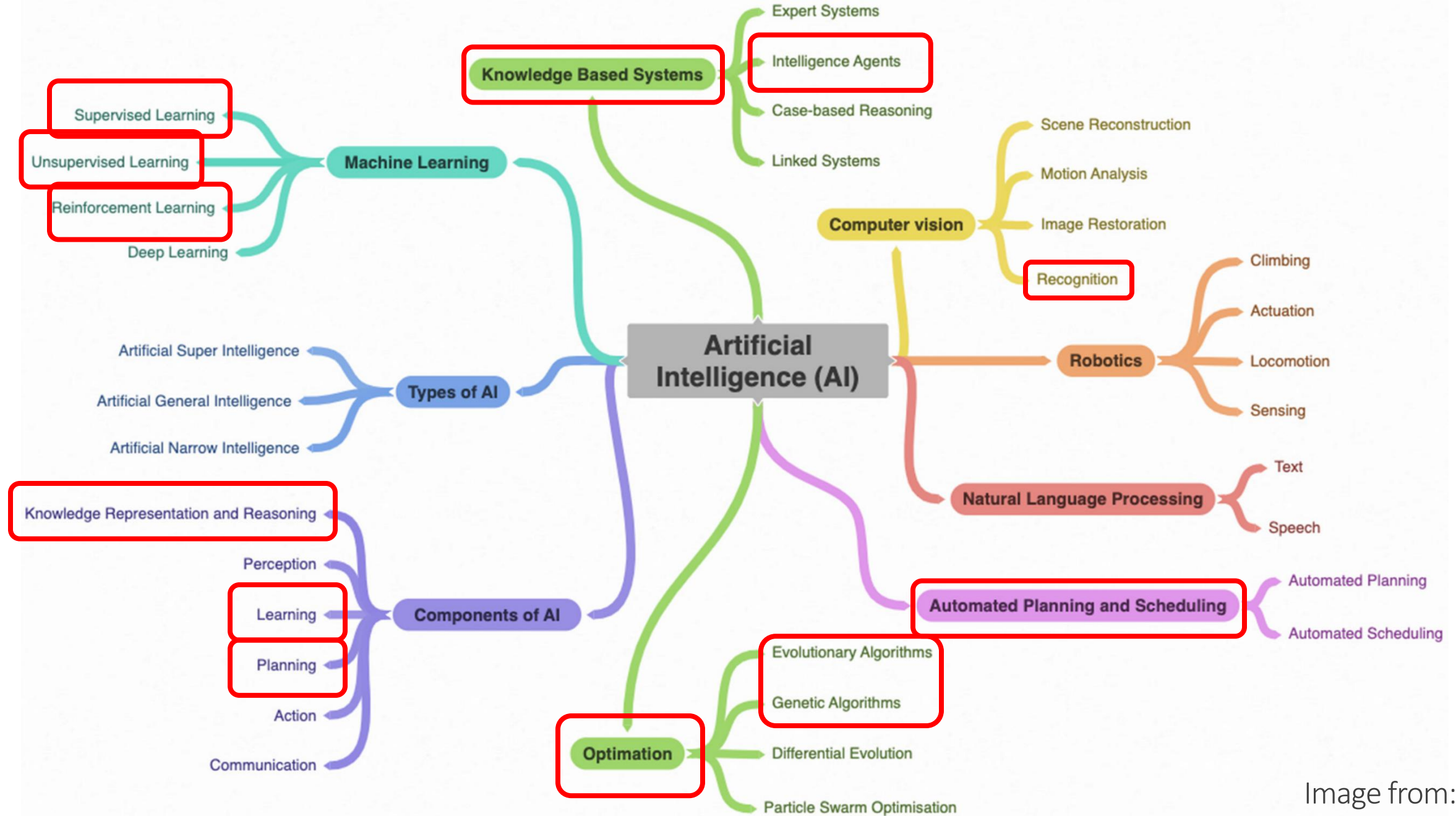


Image from: [link](#)